UNIVERSIDADE FEDERAL DE SANTA MARIA
CENTRO DE TECNOLOGIA
CURSO DE ENGENHARIA DE CONTROLE E AUTOMAÇÃO

Diogo Conterato Possebon

NORMALIZAÇÃO DE ILUMINAÇÃO DE VÍDEOS COM *U-NET* COMO PRÉ-PROCESSAMENTO PARA RASTREAMENTO DO PEIXE-ZEBRA (*Danio rerio*)

Santa Maria, RS
2025

Diogo Conterato Possebon

NORMALIZAÇÃO DE ILUMINAÇÃO DE VÍDEOS COM *U-NET* COMO PRÉ-PROCESSAMENTO PARA RASTREAMENTO DO PEIXE-ZEBRA (*Danio rerio*)

Trabalho de Conclusão de Curso apresentado ao Programa de Graduação em Engenharia de Controle e Automação da Universidade Federal de Santa Maria (UFSM, RS), como requisito parcial para obtenção de grau de Graduação em Engenharia de Controle e Automação.

Orientador: Prof. Dr. Leonardo Ramos Emmendorfer

Santa Maria, RS
2025

RESUMO

NORMALIZAÇÃO DE ILUMINAÇÃO DE VÍDEOS COM *U-NET* COMO PRÉ-PROCESSAMENTO PARA RASTREAMENTO DO PEIXE-ZEBRA (*Danio rerio*)

AUTOR: Diogo Conterato Possebon
ORIENTADOR: Prof. Dr. Leonardo Ramos Emmendorfer

Com o avanço da computação em diversas áreas da ciência, a bioinformática destaca-se ao integrar biologia, medicina, matemática, física e ciência da computação para a análise de dados biológicos. Dentro desse campo, o bioimageamento utiliza a visão computacional para capturar e analisar processos biológicos em tempo real, permitindo o rastreamento de animais e a análise de seu comportamento. Estudos com peixes são particularmente relevantes, pois o comportamento desses animais pode indicar sua saúde e bem-estar. No entanto, o rastreamento manual de peixes é um processo demorado e sujeito a erros. Além disso, os *Softwares* de rastreamento disponíveis atualmente são frequentemente complexos e difíceis de entender para cientistas da área de biologia que estudam peixes, além de não atenderem adequadamente às necessidades específicas desses pesquisadores. Em resposta a esses desafios, este trabalho propõe o desenvolvimento de um algoritmo de rastreamento de peixes em Python, em colaboração com o Laboratório de Neuropsicobiologia Experimental (LANE) da UFSM, que realiza experimentos com peixes-zebra. O objetivo geral do projeto é desenvolver um algoritmo de rastreamento de peixes que atenda às necessidades do LANE, permitindo uma análise comportamental mais eficiente e precisa.

Palavras-chave: Visão computacional. Rastreamento. Peixe-zebra (*Danio rerio*).

ABSTRACT

ILLUMINATION NORMALIZATION OF VIDEOS USING U-NET AS A PRE-PROCESSING STEP FOR ZEBRAFISH (*DANIO RERIO*) *TRACKING*

AUTHOR: Diogo Conterato Possebon
ADVISOR: Prof. Dr. Leonardo Ramos Emmendorfer

With the advancement of computing in various areas of science, bioinformatics stands out by integrating biology, medicine, mathematics, physics, and computer science for the analysis of biological data. Within this field, bioimaging utilizes computer vision to capture and analyze biological processes in real time, enabling the *tracking* of animals and the analysis of their behavior. Studies involving fish are particularly relevant, as their behavior can indicate their health and well-being. However, manual *tracking* of fish is a time-consuming process and prone to errors. Furthermore, currently available *tracking* software is often complex and difficult for biologists who study fish to understand, and it does not adequately meet their specific needs. In response to these challenges, this work proposes the development of a fish *tracking* algorithm in Python, in collaboration with the Laboratory of Experimental Neuropsychobiology (LANE) at UFSM, which conducts experiments with zebrafish. The overall objective of the project is to develop a fish *tracking* algorithm that meets the needs of LANE, enabling more efficient and accurate behavioral analysis.

Keywords: Computer vision. *Tracking*. Zebra-fish (*Danio rerio*).

Sumário

1 INTRODUÇÃO 7
1.1 MOTIVAÇÃO 8
1.2 JUSTIFICATIVA 8
1.3 FORMULAÇÃO DO PROBLEMA 9
1.4 OBJETIVOS 9
1.4.1 Objetivo Geral 9
1.4.2 Objetivos específicos 10
2 FUNDAMENTAÇÃO TEÓRICA 11
2.1 Visão computacional 11
2.1.1 Introdução a visão computacional 11
2.1.2 Aplicações da visão computacional no estudo comportamental de animais 13
2.2 Pré-processamento de imagens em visão computacional 15
2.2.1 Transformações morfológicas 15
2.3 Redes Neurais Convolucionais (*CNNs*) 16
2.3.1 Arquitetura *U-Net* 18
2.4 *Tracking* 21
2.4.1 Introdução ao *Tracking* 21
2.4.2 A importância do *tracking* e áreas de aplicação 21
2.4.3 Técnicas utilizadas 22
2.4.4 Filtragem 25
2.5 Peixe-zebra 26
3 METODOLOGIA 28
3.1 Coleta de dados 29
3.1.1 Colaboração com o Laboratório LANE 29
3.1.2 Critérios de seleção dos vídeos 29
3.1.3 Desafios na consistência da iluminação 30
3.2 Treinamento da *U-Net* para normalização de iluminação 30
3.2.1 Preparação dos dados 30
3.2.2 Pré-processamento 31
3.2.3 Arquitetura do modelo UNET 31
3.2.4 Avaliação do desempenho do modelo 32
3.3 Normalização do vídeo com *U-Net* 33
3.4 Implementação do *tracking* 34
3.4.1 Definição de Áreas de Interesse (*ROI*) 34
3.4.2 Seleção de pontos de referência do peixe 35
3.4.3 Segmentação Baseada em Cor (*HSV*) 36

3.4.4 Extração de contornos e centroide 38
3.4.5 Exportação dos dados 38
4 RESULTADOS E DISCUSSÕES 40
4.1 Testes iniciais com algoritmos de *tracking* da biblioteca *OpenCV* 40
4.1.1 Adição do filtro de *Kalman* 41
4.2 Filtro de cor *BGR* 41
4.3 Arquitetura *U-Net* 43
4.4 Rastreamento 44
4.4.1 Rastreamento sem normalização 45
4.4.2 Rastreamento *HSV* com normalização *U-Net* 46
4.4.3 Estabilidade do *tracking* 49
5 CONCLUSÃO 51
6 REFERÊNCIAS 53

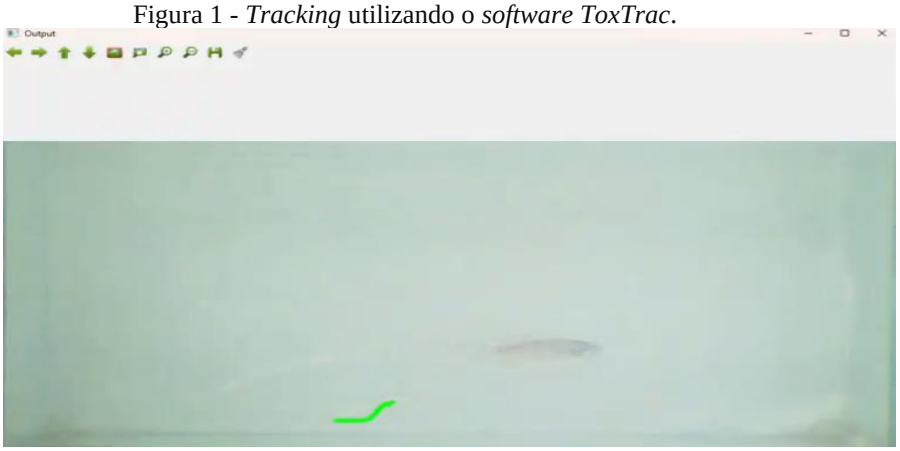
1 INTRODUÇÃO

O uso de técnicas de visão computacional tem se tornado cada vez mais relevante em estudos biológicos, especialmente para a análise automatizada de dados comportamentais obtidos a partir de vídeos experimentais. Em ambientes laboratoriais, a automação do rastreamento de organismos permite reduzir a subjetividade humana, aumentar a reprodutibilidade dos experimentos e viabilizar análises contínuas ao longo do tempo (Meijering, 2020).

O peixe-zebra (*Danio rerio*) é amplamente utilizado como modelo experimental devido à sua facilidade de manutenção, rápido desenvolvimento e elevada similaridade genética com humanos, sendo empregado em estudos de comportamento, neurobiologia e toxicologia (Kalueff et al., 2014). Em experimentos comportamentais, o rastreamento preciso do seu movimento é essencial para a extração de métricas quantitativas, como trajetória e padrões locomotores.

Métodos clássicos de rastreamento, baseados principalmente em segmentação por cor no espaço HSV, apresentam bom desempenho em condições controladas e são amplamente utilizados por sua simplicidade e baixo custo computacional (Szeliski, 2011). Entretanto, variações de iluminação, reflexos do aquário e sombras afetam diretamente a qualidade da segmentação, resultando em falhas na detecção do peixe e comprometendo a confiabilidade dos dados obtidos. A Figura 1 apresenta um exemplo de rastreamento utilizando o software ToxTrac (Rodriguez et al., 2018), enquanto a Figura 2 ilustra limitações observadas em um sistema baseado em OpenCV sob variações de iluminação.

Figura 1 - *Tracking* utilizando o *software ToxTrac*.



Fonte: Autor.

Figura 2 - *Tracking* utilizando a biblioteca do *OpenCV* em *Python*.



Fonte: Autor

Diante desse cenário, este trabalho propõe a integração de uma rede neural convolucional do tipo U-Net como etapa de pré-processamento para normalização da iluminação em vídeos experimentais. A U-Net, amplamente empregada em aplicações de bioimageamento, apresenta elevada capacidade de generalização e robustez frente a variações visuais (Ronneberger et al., 2015). O objetivo é tornar o rastreamento por cor mais estável e confiável, mantendo baixo custo computacional e viabilidade de aplicação em ambientes laboratoriais reais.

1.1 MOTIVAÇÃO

A motivação deste trabalho surge da necessidade do Laboratório de Análises Neurocomportamentais por um método de rastreamento automatizado de peixe-zebra que seja simples, robusto e de baixo custo computacional. Durante experimentos comportamentais, variações de iluminação comprometem significativamente a eficiência de métodos clássicos de rastreamento por cor, dificultando a extração confiável de dados experimentais ao longo do tempo.

1.2 JUSTIFICATIVA

A escolha da normalização de iluminação por meio de uma arquitetura U-Net como etapa de pré-processamento se justifica por sua capacidade de aprender correções espaciais

complexas de iluminação, preservando as características visuais relevantes do peixe. Diferentemente de abordagens baseadas em detectores profundos ou métodos totalmente supervisionados de rastreamento, a solução proposta mantém baixo custo computacional, reduz a necessidade de re-treinamento e pode ser facilmente integrada a pipelines clássicos de visão computacional, tornando-se adequada para uso contínuo em ambientes experimentais reais.

1.3 FORMULAÇÃO DO PROBLEMA

A análise comportamental de peixes demanda muito tempo, disponibilidade e conhecimento especializado. Isso resulta na demora na realização desses experimentos, pois é necessário analisar vários fatores relacionados à saúde dos peixes, realizar diversos testes e esperar horas até que os testes surtam efeito.

Diante deste problema, o presente trabalho propõe uma abordagem aprimorada para o rastreamento automatizado de peixes-zebra, integrando uma rede neural convolucional do tipo *U-Net* como etapa de normalização de iluminação no pré-processamento das imagens, seguida de rastreamento em *HSV*. Essa estratégia visa mitigar os efeitos de iluminação variável, mantendo a leveza computacional necessária para análises em tempo real, ao mesmo tempo que melhora a precisão da segmentação. Com isso, espera-se fornecer aos pesquisadores uma ferramenta mais robusta e confiável para análise automatizada do comportamento de peixes em ambiente laboratorial.

1.4 OBJETIVOS

Visando solucionar o problema proposto acima, foram definidos objetivos para o trabalho, divididos em geral e específicos, sendo estes apresentados abaixo.

1.4.1 Objetivo Geral

Desenvolver um sistema de rastreamento de peixes-zebra mais robusto à variação de iluminação, utilizando normalização por U-Net como etapa de pré-processamento.

1.4.2 Objetivos específicos

- Compreender e analisar as necessidades do laboratório LANE no rastreamento de peixes-zebra, com ênfase nos desafios causados por variações de iluminação durante os experimentos;
- Desenvolver um algoritmo de rastreamento em Python, integrando técnicas de normalização de iluminação por meio de redes neurais convolucionais (*U-Net*), a fim de garantir maior robustez e precisão na detecção e acompanhamento dos peixes em diferentes condições experimentais.

2 FUNDAMENTAÇÃO TEÓRICA

O objetivo deste capítulo é abranger de forma geral os conteúdos do projeto apresentado. Ele será dividido em uma introdução a visão computacional, estudos sobre o *tracking*, e características gerais do peixe-zebra (*Danio rerio*).

2.1 Visão computacional

O objetivo desta seção é proporcionar ao leitor uma compreensão abrangente da visão computacional e suas aplicações no estudo do comportamento animal.

2.1.1 Introdução a visão computacional

A visão computacional é uma área da ciência que habilita máquinas a interpretar e compreender o ambiente visual ao seu redor por meio da análise de imagens capturadas por câmeras, sensores, scanners e outros dispositivos digitais. Seu principal objetivo é emular a capacidade humana de percepção visual, utilizando imagens como entrada para gerar interpretações, seja da cena completa ou de elementos específicos dentro dela. Essa capacidade envolve uma série de desafios complexos, pois a percepção humana integra aquisição, processamento e interpretação das informações visuais de forma altamente eficiente e adaptativa, o que impulsiona o desenvolvimento contínuo de algoritmos avançados para análise de imagens (Pedrini e Schwartz, 2008).

O processamento digital de imagens pode ser dividido em etapas fundamentais, incluindo aquisição, pré-processamento, segmentação, representação, descrição, reconhecimento e interpretação. A etapa de aquisição refere-se à captura das imagens por dispositivos físicos, enquanto o pré-processamento abrange técnicas como redução de ruído, correção de contraste e brilho, suavização e normalização das propriedades visuais da imagem, visando preparar os dados para análises subsequentes (Zhang et al., 2017). A segmentação é um passo crítico para identificar e separar regiões de interesse dentro da imagem, utilizando critérios como bordas, texturas e si*mil*aridade de pixels (Shi et al., 2020). Em seguida, a representação e descrição estruturam e extraem características relevantes dos objetos segmentados, permitindo a classificação e reconhecimento por meio da atribuição de rótulos

semânticos. Por fim, a interpretação confere significado às informações extraídas, possibilitando tomadas de decisão automatizadas ou suporte à análise humana (Pedrini e Schwartz, 2008).

Paralelamente ao processamento bidimensional, avanços significativos foram obtidos no desenvolvimento de técnicas para reconstrução tridimensional de ambientes e objetos a partir de múltiplas imagens, contribuindo para aplicações que demandam percepção espacial detalhada. Modelos 3D podem ser construídos com alta precisão a partir da combinação de milhares de fotografias parcialmente sobrepostas, como exemplificado pelo processo de *morphing* apresentado na Figura 3(Gomes, Darsa et al., 1999).

Figura 3 - *Morphing*: mistura entre duas fotografias



Fonte: (Gomes, Darsa et al. 1999)

A visão computacional é um problema intrinsecamente inverso e desafiador, pois requer a recuperação de informações e variáveis ocultas a partir de dados incompletos e muitas vezes ambíguos. Para solucionar esses problemas, é necessário o uso de modelos baseados em princípios físicos, probabilísticos e, cada vez mais, técnicas de aprendizado de máquina que utilizam grandes conjuntos de dados para inferir representações robustas (LeCun, Bengio & Hinton, 2015). Modelar o mundo visual, com sua complexidade e diversidade, é muito mais complexo do que outros domínios, como o processamento de fala, devido à variabilidade das condições de iluminação, geometria, texturas e oclusões (Gomes, Darsa et al., 1999).

As bases teóricas da visão computacional envolvem áreas como física (radiometria, óptica), design de sensores e gráficos computacionais, que modelam como a luz interage com os objetos, como ela é transmitida e refletida até o sensor, e como essa informação é projetada no plano da imagem. Embora os gráficos computacionais ainda evoluam, já permitem renderizações realistas para cenas estáticas e animações complexas (Szeliski, 2022).

Um dos maiores desafios em visão computacional aplicada a imagens biológicas e experimentais é a variação da iluminação durante a captura das imagens, que pode afetar a qualidade da segmentação e rastreamento (Zheng et al., 2021). Técnicas modernas têm aplicado redes neurais convolucionais (*CNNs*), incluindo arquiteturas do tipo *U-Net*, para normalização e realce de imagens, melhorando a robustez dos sistemas contra ruídos e variações locais de brilho e contraste (Ronneberger et al., 2015; Isola et al., 2017). Essas abordagens possibilitam

pré-processamento eficaz em vídeos e imagens, facilitando a posterior segmentação e rastreamento em condições variáveis de iluminação (Fu et al., 2020).

O objetivo central da visão computacional é extrair informações relevantes e úteis a partir de dados visuais brutos, habilitando tarefas como reconhecimento de objetos, detecção e rastreamento de movimentos, reconstrução 3D, análise de cenas e compreensão contextual. Essas capacidades têm aplicações abrangentes em medicina, robótica, automação industrial, segurança, entretenimento, ecologia e biologia experimental, especialmente na análise comportamental de organismos vivos. Entretanto, desafios persistentes, como variações imprevisíveis de iluminação, oclusões parciais, deformações e ruídos nas imagens, tornam o desenvolvimento de sistemas robustos uma tarefa complexa, exigindo soluções que integrem pesquisa teórica e inovação tecnológica para atender às demandas específicas de cada aplicação (Pedrini e Schwartz, 2008; Szeliski, 2022).

2.1.2 Aplicações da visão computacional no estudo comportamental de animais

A visão computacional tem revolucionado o campo do estudo comportamental de animais, permitindo aos pesquisadores obter e analisar dados com uma precisão e eficiência sem precedentes. Essa tecnologia se baseia na utilização de algoritmos e técnicas avançadas para capturar, processar e interpretar informações visuais de forma automatizada, transformando a maneira como os cientistas observam e compreendem o comportamento animal em diversos contextos.

No trabalho de Koniar et al., (2016), os autores enfrentaram o desafio de rastrear porquinhos-da-índia em condições naturais, sem modificações prévias na cena, como a presença de sombras, reflexos e iluminação não homogênea. Para garantir a robustez dos algoritmos de detecção de movimento, especialmente aqueles baseados em limiarização, foi utilizada a técnica de limiarização por entropia. Embora essa técnica tenha sido eficaz em algumas condições, ela apresentou dificuldades em situações de iluminação precária, onde a detecção do porquinho-da-índia foi limitada a áreas bem iluminadas da gaiola. Para mitigar esses desafios, foi implementado uma correção de fundo adaptativa, que melhorou a precisão do rastreamento em ambientes com variação de luz, especialmente em condições noturnas. Além disso, métodos diferenciais, como a subtração de quadros consecutivos e a remoção de fundo estático, foram usados para lidar com a estabilidade dos resultados em diferentes

cenários, demonstrando serem eficazes mesmo quando o porquinho-da-índia estava parcialmente oculto ou em movimento.

Um aspecto interessante deste estudo foi o uso da correspondência de cores, que, embora computacionalmente mais exigente, provou ser útil no rastreamento offline, especialmente ao subtrair a imagem da cena vazia para eliminar detecções falso-positivas. Os resultados indicaram que, apesar das variações no movimento e nas condições de iluminação, os métodos diferenciais e a correção de fundo adaptativa proporcionaram os melhores resultados em termos de precisão de rastreamento.

O artigo (Janku et al., 2016) destaca a dificuldade de criar um algoritmo de rastreamento universal e enfatiza que a escolha do algoritmo adequado depende não apenas do princípio do algoritmo, mas também da implementação específica, incluindo linguagem de programação, compiladores e otimizações manuais. Neste artigo, foram avaliadas diversas funções de rastreamento da biblioteca *OpenCV*, incluindo *TLD*, *SURF* e *SIRF*. As conclusões destacam que, apesar da sofisticação dos algoritmos, a implementação e as otimizações são fatores cruciais para a performance e robustez do rastreamento. Os resultados mostraram que o *TLD*, embora considerado o melhor em pesquisas anteriores, teve um desempenho inferior ao esperado devido à sua complexidade e à implementação inadequada na *OpenCV*. Além disso, o algoritmo *SIRF* foi identificado como o mais lento, seguido de perto pelo *SURF* e pelo *TLD*.

De acordo com (Soleimanitaleb et al., 2019) o aprendizado profundo é um campo novo e empolgante, este que tem mostrado alta precisão e feito grandes avanços em várias áreas, especialmente na visão computacional. Redes de aprendizado profundo possuem muitas camadas que extraem diferentes características do objeto em cada camada, proporcionando uma boa precisão no rastreamento de objetos devido à complexidade do modelo, que é mais lenta em comparação com redes rasas. No entanto, não se pode afirmar que o aprendizado profundo é a melhor solução para todos os casos e deve sempre ser utilizado. Conhecendo as vantagens e desvantagens de todos os métodos, é possível determinar qual abordagem pode ser mais eficaz na resolução de problemas específicos. O aprendizado de máquina, embora muito eficiente em muitas aplicações, pode não ser a melhor opção em todos os cenários, e essa complexidade deve ser considerada ao selecionar a abordagem mais adequada.

2.2 Pré-processamento de imagens em visão computacional

O pré-processamento de imagens é uma etapa essencial em sistemas de visão computacional, especialmente em contextos aplicados à biologia, como a análise comportamental de peixes-zebra. Essa etapa visa preparar os dados visuais brutos para tarefas posteriores, como segmentação, rastreamento e classificação, corrigindo distorções causadas por ruído, iluminação irregular, baixa resolução, entre outros fatores que podem comprometer o desempenho dos algoritmos subsequentes (Gonzalez & Woods, 2018).

2.2.1 Transformações morfológicas

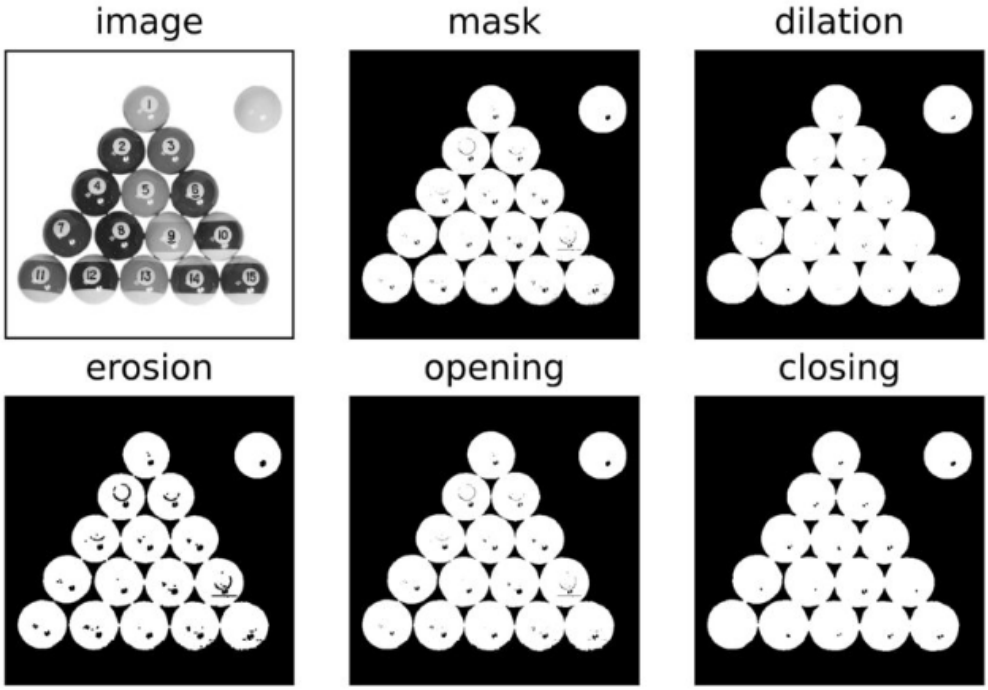
Transformações morfológicas são operações fundamentais no processamento de imagens binárias, utilizadas para refinar resultados de segmentação, reduzir ruídos e melhorar a coerência das regiões detectadas. Essas operações atuam sobre a forma dos objetos na imagem por meio de um elemento estruturante aplicado à vizinhança dos pixels.

Entre as operações mais utilizadas estão a erosão, que remove pixels das bordas e elimina pequenas regiões isoladas, e a dilatação, que expande os objetos e conecta áreas fragmentadas. A abertura (erosão seguida de dilatação) é empregada para remoção de ruídos, enquanto o fechamento (dilatação seguida de erosão) auxilia no preenchimento de lacunas internas.

Essas técnicas são amplamente aplicadas como pós-processamento após a segmentação, seja por métodos clássicos ou por redes neurais, contribuindo para a geração de máscaras mais estáveis e representativas da morfologia real dos objetos, como peixes em aquários experimentais (Adhikari et al., 2017; Meijering, 2020).

A Figura 4 ilustra os efeitos das principais transformações morfológicas aplicadas sobre uma imagem binária, evidenciando como cada operação altera a estrutura dos objetos segmentados (DataHacker, 2020).

Figura 4 - Exemplo de aplicação de máscara e operações morfológicas (dilatação, erosão, abertura e fechamento).



Fonte: DataHacker (2020). Disponível em: https://datahacker.rs/006-morphological-transformations-with-OpenCV-in-python/.

2.3 Redes Neurais Convolucionais (*CNNs*)

As redes neurais convolucionais (*CNNs*) representam a base da visão computacional moderna, sendo especialmente eficazes na extração de características hierárquicas de imagens, desde padrões de baixo nível, como bordas e texturas, até representações semânticas complexas (Goodfellow et al., 2016; LeCun et al., 2015). Tradicionalmente aplicadas à classificação e detecção, as *CNNs* têm se mostrado extremamente versáteis em tarefas de pré-processamento de imagens, incluindo remoção de ruído, restauração, equalização de contraste e, particularmente, normalização de iluminação (Xu et al., 2017; Li et al., 2021).

A normalização de iluminação é um desafio comum em imagens biológicas e biomédicas, onde condições de captura não uniformes podem comprometer a análise automatizada de estruturas ou organismos. Métodos tradicionais, como equalização de histograma ou filtragem adaptativa, frequentemente introduzem artefatos ou alteram informações morfológicas críticas. Nesse contexto, *CNNs* oferecem a vantagem de aprender mapeamentos complexos entre imagens distorcidas e suas versões ideais, preservando detalhes estruturais enquanto corrigem variações de brilho e contraste (Cai et al., 2019; Zhang et al., 2020).

Diversas arquiteturas de *CNN* têm sido exploradas para esse propósito, são elas:

- *U-Net* (Ronneberger et al., 2015; Falk et al., 2019): Inicialmente projetada para segmentação de imagens biomédicas, a *U-Net* mostrou-se eficaz em normalização de iluminação ao capturar simultaneamente informações globais e locais por meio de sua estrutura simétrica de *encoder-decoder* com *skip connections*. Essa abordagem permite à rede restaurar imagens com distorções de brilho sem comprometer detalhes estruturais essenciais, sendo aplicada em microscopia, imagens de peixes-zebra e histologia digital (Li et al., 2021; Zimmerer et al., 2018).
- *ResUNet* (Xiao et al., 2018; Zhang et al., 2021): Combina blocos residuais inspirados na *ResNet* com a estrutura *U-Net*, melhorando o fluxo de gradiente em redes profundas. Essa característica é especialmente útil em tarefas de normalização de iluminação em imagens de alta resolução, permitindo um treinamento mais estável e preservando texturas finas.
- Attention *U-Net* (Oktay et al., 2018; Zhou et al., 2021): Integra mecanismos de atenção para que a rede foque em regiões de interesse mais críticas durante a restauração, evitando alterações em áreas de fundo ou estruturas irrelevantes. Essa abordagem é valiosa em cenários onde partes do objeto podem apresentar sombras ou reflexos localizados.
- *SegNet* (Badrinarayanan et al., 2017): Arquitetura *encoder-decoder* que utiliza índices de *pooling* armazenados no *encoder* para realizar *upsampling* no *decoder*, preservando a localização espacial das características. No contexto de normalização de iluminação, a *SegNet* consegue corrigir variações de brilho e contraste, mantendo contornos e texturas importantes, sendo uma alternativa eficiente quando se deseja reduzir o custo computacional em imagens de alta resolução.

A escolha da *U-Net* como arquitetura central para normalização de iluminação se justifica por sua capacidade de combinar informações de contexto global com detalhes locais, facilitando a restauração de imagens em ambientes com iluminação não uniforme, como aquários ou cenas biológicas microscópicas. Além disso, variantes como *U-Net++* ou *Attention U-Net* podem ser adaptadas dependendo da complexidade do padrão de iluminação e do nível de detalhe necessário (Zhou et al., 2018; Oktay et al., 2018).

Estudos recentes confirmam a eficácia das *CNNs* para normalização de iluminação em imagens biomédicas e biológicas. Por exemplo, (Li et al., 2021) demonstraram que redes baseadas em *U-Net* conseguem restaurar imagens de peixes-zebra capturadas sob diferentes condições de luz, melhorando significativamente o desempenho de etapas subsequentes, como rastreamento, segmentação e análise comportamental. De forma semelhante, (Zhang et al.,

2020) mostraram que modelos residuais e baseados em atenção preservam contornos e detalhes morfológicos durante a equalização de brilho, superando métodos convencionais de pré-processamento.

Dessa forma, as *CNNs* não apenas fornecem soluções robustas para normalização de iluminação, mas também estabelecem uma base confiável para fluxos de processamento subsequentes, garantindo que análises automatizadas de imagens biológicas sejam precisas, reprodutíveis e menos sensíveis a condições de captura variáveis.

2.3.1 Arquitetura *U-Net*

A *U-Net* é uma arquitetura de rede neural convolucional desenvolvida especificamente para segmentação de imagens biomédicas, sendo apresentada inicialmente por (Ronneberger et al., 2015)). Essa proposta surgiu diante da necessidade de realizar segmentações precisas mesmo com conjuntos de dados rotulados relativamente pequenos — uma limitação comum em aplicações médicas e biológicas. Desde então, a *U-Net* tem sido amplamente utilizada e adaptada em uma variedade de tarefas de processamento de imagens, como normalização de iluminação, reconstrução de imagens, remoção de ruído e realce de estruturas específicas, especialmente em domínios como biologia computacional, histologia digital, zoologia, e imagens em tempo real de microscopia (Falk et al., 2019; Zimmerer et al., 2018).

A arquitetura da *U-Net* é caracterizada por seu formato em "U", composta por duas partes principais que operam de forma simétrica: o caminho de contração (*encoder*) e o caminho de expansão (*decoder*).

O *encoder* é responsável por capturar o contexto global da imagem. Ele consiste em uma sequência de blocos convolucionais, seguidos de operações de *pooling* — tipicamente *max pooling* — que reduzem progressivamente a resolução espacial da imagem, ao mesmo tempo em que aumentam a profundidade das representações extraídas (Ronneberger et al., 2015). Cada bloco geralmente contém duas camadas convolucionais com ativação *ReLU*, podendo incluir normalização por *batch* (*batch normalization*), seguida de uma operação de *downsampling*. Essa estrutura permite à rede aprender representações latentes robustas, capazes de codificar com precisão os aspectos semânticos relevantes da imagem.

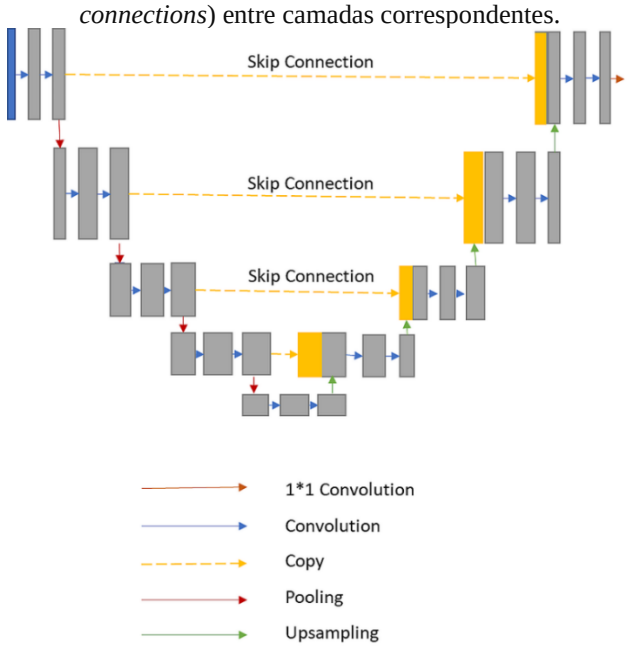
O *decoder* tem como objetivo reconstruir a resolução espacial da imagem original a partir das representações latentes extraídas no *encoder*. Para isso, são utilizadas camadas de

upsampling (como as convoluções transpostas) que aumentam gradualmente a dimensão espacial das ativações (Çiçek et al., 2016).

Após cada etapa de *upsampling*, a *U-Net* realiza a concatenação das ativações correspondentes do *encoder* por meio de *skip connections*. Essas conexões simétricas transferem diretamente informações espaciais finas para o *decoder*, compensando as perdas causadas pelas operações de *pooling*. Essa estratégia garante a preservação de detalhes morfológicos importantes, o que é essencial em tarefas de segmentação de precisão, como a identificação de contornos celulares, estruturas anatômicas ou organismos inteiros em ambientes naturais.

A simetria entre o *encoder* e o *decoder*, unida ao uso estratégico das *skip connections*, torna a *U-Net* extremamente eficaz mesmo quando treinada com um número limitado de amostras, fator que contribuiu para sua rápida adoção em diversas áreas das ciências da vida. Na Figura 5, é apresentado um fluxograma esquemático da arquitetura *U-Net*, destacando a organização simétrica dos módulos e o papel das conexões de atalho na transmissão de informações entre as partes da rede:

Figura 5 - Arquitetura *U-Net* com caminho de contração, caminho de expansão e conexões de salto (*skip connections*) entre camadas correspondentes.



Fonte: (THIAGARAJAN et al., 2024)

A figura acima ilustra o fluxo de dados da *U-Net* da esquerda para a direita, com destaque para as conexões de salto entre camadas de mesmo nível hierárquico. A combinação dos caminhos de contração e expansão com as *skip connections* permite que a rede atue de forma eficiente tanto na extração de contexto quanto na localização precisa dos objetos segmentados. Essa abordagem, embora originalmente criada para segmentação de imagens

biomédicas, mostrou-se adaptável a diferentes tarefas em visão computacional, incluindo detecção de bordas, mapeamento semântico e normalização de imagens.

Matematicamente, a saída *Y* da *U-Net* pode ser representada como:

$$Y = f_{decoder}\left(f_{encoder}(X)\right) + \sum_{i=1}^{n} skip_i$$

onde $f_{encoder}$ e $f_{decoder}$ são as funções aprendidas pelos caminhos de contração e expansão, respectivamente, $skip_i$ representa os mapas de ativação conectados entre camadas espelhadas, e *X* é a imagem de entrada. A soma com as *skip connections* é frequentemente implementada como concatenação seguida de convoluções adicionais, ao invés de soma direta, a fim de preservar a diversidade de características (Milletari et al., 2016).

Desde sua introdução, diversas variantes da *U-Net* foram propostas para melhorar seu desempenho em diferentes cenários. A *U-Net++* (Zhou et al., 2018), por exemplo, introduz caminhos densos entre as camadas do *encoder* e *decoder*, com o objetivo de reduzir o *gap* semântico entre as conexões de salto. Já a *Attention U-Net* (Oktay et al., 2018) incorpora mecanismos de atenção para que a rede aprenda a focar nas regiões mais relevantes da imagem durante a segmentação. Outra evolução notável é a *ResUNet*, que substitui os blocos convolucionais convencionais por blocos residuais inspirados na *ResNet*, promovendo um melhor fluxo de gradiente e mais estabilidade durante o treinamento (Xiao et al., 2018).

Em experimentos recentes de normalização de iluminação com imagens biológicas, como em peixes-zebra (*Danio rerio*), a *U-Net* tem se mostrado capaz de preservar informações estruturais mesmo após a remoção de distorções de brilho, contrastando com métodos convencionais de equalização de histograma. Trabalhos como os de (Li et al., 2021) demonstraram que redes baseadas em *U-Net*, quando treinadas com pares de imagens distorcidas e suas versões ideais, conseguem restaurar condições visuais mais próximas das naturais, otimizando etapas subsequentes como rastreamento, contagem e segmentação de indivíduos em movimento.

Portanto, a *U-Net* não apenas permanece relevante como modelo de segmentação, mas também como ferramenta versátil para pré-processamento de imagens, com aplicações que vão além da segmentação, estendendo-se para restauração, reconstrução e normalização em contextos científicos diversos.

2.4 *Tracking*

O objetivo desta seção é fornecer ao leitor uma compreensão abrangente do *tracking*, incluindo suas aplicações e técnicas fundamentais.

2.4.1 Introdução ao *Tracking*

O *tracking* ou rastreamento é uma das aplicações mais empregadas da visão computacional quando se trata de análise de vídeos (SZELISKI, 2022). A técnica de *tracking* envolve o seguimento contínuo de objetos em movimento ao longo do tempo em uma sequência de imagens ou vídeos. A fase de *tracking* é composta essencialmente por duas etapas: detecção e correspondência temporal. Na detecção, busca-se extrair informações sobre a posição dos objetos de interesse a partir das imagens capturadas. Dependendo dos objetivos, a detecção pode fornecer outras informações sobre os objetos, atribuindo a cada um uma identidade única. Já a correspondência temporal tem como objetivo estabelecer uma ligação entre os objetos em diferentes imagens da sequência de vídeo, mantendo a mesma identidade atribuída na imagem anterior. Isso permite registrar o movimento dos objetos ao longo da cena (RODRIGUES, 2012). O *tracking* permite uma compreensão mais detalhada do movimento, comportamento e interações dos objetos em um ambiente em constante mudança.

2.4.2 A importância do *tracking* e áreas de aplicação

O *tracking* possui uma ampla área de aplicação devido à sua capacidade de facilitar análises detalhadas em diversas áreas. Ele possibilita uma eficiente análise posterior, seja para estudar doenças e condições de saúde em animais, monitorar processos industriais para garantir sua eficiência máxima, avaliar o estado de saúde de pacientes na medicina, assegurar a segurança de objetos valiosos durante o transporte, ou fornecer atualizações em tempo real sobre o status de encomendas online. Essa versatilidade torna o *tracking* uma ferramenta indispensável em uma ampla gama de cenários, impulsionando a eficácia, a segurança e o controle em diversas áreas de aplicação.

2.4.3 Técnicas utilizadas

Nesta seção, serão apresentadas as técnicas utilizadas para o desenvolvimento do projeto e a realização dos testes.

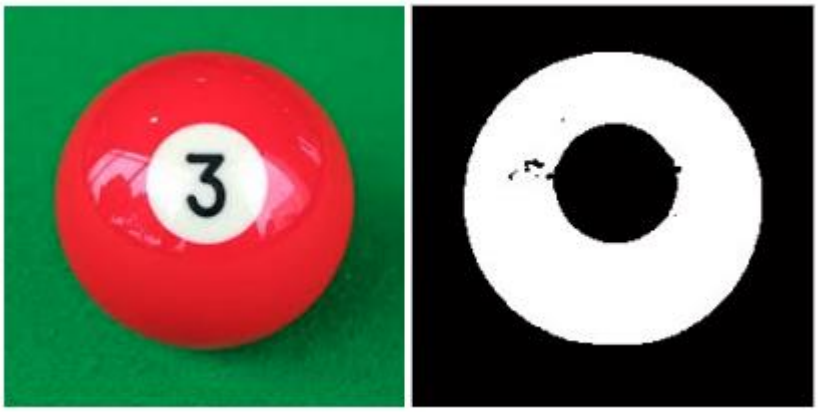
2.4.3.1 Segmentação de cores

A cor é facilmente obtida e relativamente estável em face de variações de perspectiva, ao contrário de outras características de imagem. O baixo custo computacional dos algoritmos propostos torna a cor uma característica desejável para explorar quando apropriado. Em tempo real, para detecção e rastreamento de veículos, utiliza-se um algoritmo baseado em histograma de cor. Para segmentar a imagem em plano de fundo e objeto e descrever a distribuição de cor dentro da sequência de imagens, um modelo é construído utilizando o método de Mistura *Gaussiana*. O buffer de oclusão é empregado para lidar com a oclusão do objeto (Balaji; Karthikeyan, 2017).

2.4.3.1.1 Detecção de cores

A identificação e o rastreamento por cor são realizados comparando os parâmetros dos pixels da imagem no espaço de cor adotado com uma faixa de valores especificada pelo usuário. Após a comparação, é gerada uma matriz binária a partir da imagem original. Nesta matriz, os pixels que se encaixam na faixa de valores desejada são representados pelo valor binário "1", enquanto os demais são substituídos por "0", criando um mapa de *bits*. Na representação binária, os *bits* com valor "1" aparecem em branco, e os *bits* com valor "0" aparecem em preto. No processo de reconhecimento de cores, as áreas correspondentes às cores reconhecidas surgem como regiões brancas, comumente chamadas de *blobs* (Santos Silva et al., 2016). A Figura 6 mostra a imagem original e a matriz de *bits* resultante do reconhecimento da cor vermelha.

Figura 6 - Imagem original e o mapa de *bits* correspondente.

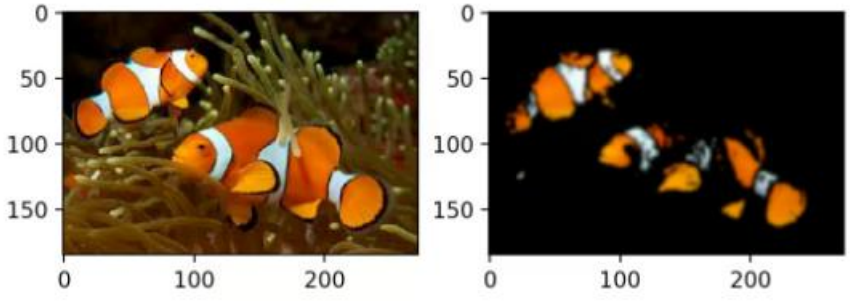


Fonte: (Santos Silva et al., 2016).

2.4.3.1.2 Problemas da Segmentação de cores

No entanto, as modelagens clássicas existentes na literatura para o tratamento de cores em imagens têm se mostrado falhas em muitos aspectos (BALLARD; BROWN, 1982), principalmente naqueles onde as condições do ambiente, particularmente a iluminação, não são as ideais ou nem sempre permanecem constantes, como é comum no contexto de futebol de robôs móveis autônomos (REALI COSTA et. al., 1999). Outro grande problema da segmentação de cores é o ruído gerado quando o ambiente possui faixas de cores *RGB* muito semelhantes ao objeto que deve ser rastreado. Isso pode dificultar a distinção precisa entre o objeto de interesse e o fundo, resultando em erros no rastreamento e reduzindo a eficácia da técnica. Na Figura 7, podemos observar um exemplo em que a faixa de cores do ambiente e do peixe são semelhantes ilustrando esse problema.

Figura 7 - Erro na detecção de cores.



Fonte: https://realpython.com/python-*OpenCV*-color-spaces/.

2.4.3.2 *Softwares*

Softwares de rastreamento estão disponíveis tanto como ferramentas gratuitas quanto comerciais. Os *Softwares* comerciais geralmente oferecem mais recursos e flexibilidade, especialmente no que se refere aos formatos de vídeo de entrada e aos resultados estatísticos. No entanto, muitos pesquisadores não podem pagar por essas ferramentas caras, e os algoritmos utilizados por esses *Softwares* frequentemente não são transparentes para o usuário. Portanto, pesquisadores costumam utilizar *Softwares* gratuitos. A cada ano, são publicados vários novos tipos e versões de *Softwares*. O lançamento contínuo de novos *Softwares* torna desafiador selecionar o mais adequado para um experimento específico ou compreender as limitações e diferenças de cada opção. Por exemplo, muitos aplicativos utilizam o mesmo pipeline e técnicas de processamento semelhantes, mas pode haver etapas no algoritmo que sejam limitantes para certos tipos de experimentos. Além disso, alguns programas têm capacidades e funções específicas limitadas ao que podem realizar e analisar (Panadeiro et. al., 2021).

Nos *Softwares* de aquisição de vídeo, a limitação mais agravante é a velocidade de processamento e a memória necessária para executar os algoritmos de rastreamento. Esta limitação é o maior problema em experimentos comportamentais. Para isso, é necessário que a máquina tenha capacidade de processar vídeos de alta resolução e alta taxa de quadros. A capacidade de processar vídeos de alta resolução e alta taxa de quadros em tempo real (análise online) revolucionaria os experimentos comportamentais, reduzindo significativamente a escala de tempo de análise e diminuindo a necessidade de armazenamento de dados de vídeo. No entanto, atualmente, a realização de rastreamento em tempo real só é possível usando algoritmos simples que não funcionam com fundos complexos e múltiplos alvos. A capacidade de realizar análises detalhadas e precisas é crucial, mas enfrenta desafios devido às exigências computacionais (Panadeiro et. al., 2021).

As técnicas utilizadas pelos *Softwares* para subtração de fundo são divididas em duas categorias principais: técnicas estáticas e técnicas dinâmicas. As técnicas estáticas incluem a utilização de um quadro de referência ou vídeo do fundo sem animais e a estimativa de um fundo pela média dos quadros do vídeo. Essas técnicas são eficazes quando os objetos de fundo e a iluminação permanecem constantes durante o experimento, sendo fáceis de implementar para detectar animais estacionários ou em movimento. Por outro lado, as técnicas dinâmicas utilizam modelos de média móvel ou decaimento e modelos de mistura *Gaussiana*. Essas abordagens são capazes de lidar com mudanças na iluminação e outras alterações graduais no

fundo, sendo importantes para experimentos de longa duração onde o sol é usado como fonte de luz (Panadeiro et. al., 2021).

No entanto, elas não são confiáveis para detectar animais que permanecem estáticos durante uma parte substancial do experimento. Além dessas técnicas, há uma abordagem inovadora utilizada pelo software *Mouse Tracking*, que emprega um algoritmo de aprendizado profundo para separar os pixels do fundo e do primeiro plano. Esta estratégia é mais robusta do que as outras técnicas, mas também é complexa e requer uma grande quantidade de dados de treinamento, o que a torna muito pesada computacionalmente.

2.4.4 Filtragem

Nesta seção, serão apresentadas técnicas de filtragem utilizadas para o desenvolvimento do projeto e a realização dos testes.

2.4.4.1 Filtro de *Kalman*

O filtro de *Kalman* estima o estado de um sistema dinâmico, mesmo que a forma exata do sistema não seja conhecida. Este filtro é extremamente poderoso, pois permite estimativas de estados passados e futuros. A descrição do filtro de *Kalman* padrão e seus algoritmos abrange duas etapas principais: a etapa de predição e a etapa de correção.

Quando aplicado ao rastreamento, o filtro de *Kalman* funciona estimando a posição e a velocidade de um objeto em movimento a partir de medições ruidosas. O processo de rastreamento com o filtro de *Kalman* envolve a predição do próximo estado do objeto (posição e velocidade) com base em seu estado atual e na dinâmica do sistema, seguida pela correção dessa predição utilizando novas medições obtidas. Esse ciclo de predição e correção permite ao filtro de *Kalman* rastrear objetos de forma precisa e robusta, mesmo na presença de incertezas e ruídos nas medições.

2.4.4.2 Filtro de *Kernel*

O filtro de *Kernel* é uma técnica de processamento de imagens que envolve a aplicação de uma matriz, conhecida como *Kernel*, sobre a imagem para produzir uma versão filtrada. O *Kernel* é uma pequena matriz de números que é usada para realizar operações como suavização,

nitidez, detecção de bordas e remoção de ruído. O *Kernel* é convolvido com a imagem original, ou seja, a matriz é movida através de cada pixel da imagem, aplicando operações matemáticas que resultam na modificação dos valores dos pixels

No projeto, abordaremos dois tipos específicos de filtros de *Kernel*: o filtro *gaussiano* e o filtro de média de *Kernel*.

2.4.4.2.1 Filtro de *Kernel* de média

Um *Kernel* de média é uma matriz utilizada no processamento de imagens para suavizar ou desfocar a imagem. Esse tipo de *Kernel* ajuda a reduzir o ruído e os detalhes finos, tornando a imagem mais "suave". O rastreamento por *Kernel* é realizado calculando o movimento do objeto alvo utilizando uma representação primitiva da região do objeto. Para evitar a necessidade de uma busca exaustiva, foram propostas abordagens mais eficientes que restringem a busca do objeto à vizinhança de sua posição anterior (Schweitzer et. al., 2002)

A suavização pode ser útil em várias tarefas de visão computacional, como a detecção de bordas e contornos. Quando aplicamos um *Kernel* de média, cada pixel da imagem é substituído pela média dos valores dos pixels na sua vizinhança. Para um *Kernel* de matriz 5x5 por exemplo, significa que para cada pixel na imagem original, consideramos uma vizinhança de 5x5 pixels ao redor desse pixel. Os valores desses 25 pixels são somados e divididos pelo número total de pixels (25), e o valor resultante substitui o valor original do pixel central. O *Kernel* de média funciona deslocando essa matriz 5x5 sobre cada pixel da imagem original, aplicando o mesmo processo. Nas bordas da imagem, onde não há pixels suficientes para completar a matriz de 5x5, a implementação pode ignorar os pixels fora da borda ou replicar os valores dos pixels na borda.

2.5 Peixe-zebra

O peixe-zebra (*Danio rerio*) demonstrado na Figura 8 tem se tornado uma escolha popular na pesquisa translacional em neurociência e comportamento. Suas semelhanças fisiológicas com os mamíferos, facilidade para manipulações genéticas, alta sensibilidade a fatores farmacológicos e genéticos, comportamento robusto, baixo custo e potencial para

triagem em larga escala são fatores que impulsionam o uso crescente desse modelo. A pequena dimensão física e o ciclo de vida rápido do peixe-zebra permitem estudos de desenvolvimento acelerados, possibilitando a observação de efeitos genéticos e farmacológicos ao longo de várias gerações em um período curto de tempo. Estudar os fenótipos comportamentais do peixe-zebra oferece importantes insights sobre as vias neurais, biomarcadores fisiológicos e bases genéticas do funcionamento cerebral, tanto em condições normais quanto patológicas (Kalueff et. al., 2013).

Figura 8 - Peixe Zebra (*Danio rerio*).



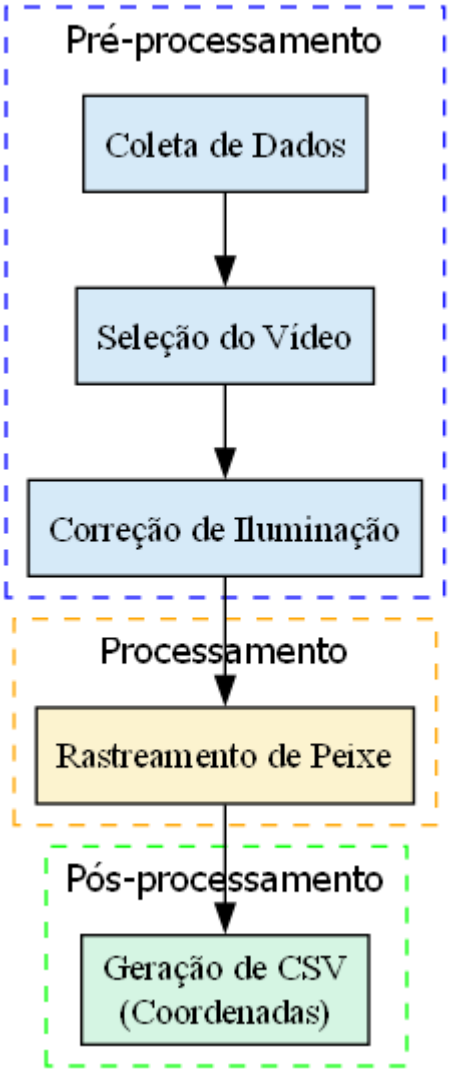
Fonte - https://www.petz.com.br/blog/curiosidades/peixe-zebra/.

A facilidade de visualização e manipulação do sistema nervoso do peixe-zebra, devido à sua transparência durante os estágios iniciais de desenvolvimento, proporciona uma plataforma poderosa para investigar a neurobiologia do comportamento. Além disso, a rápida evolução de novos paradigmas comportamentais com peixe-zebra destaca a necessidade de uma terminologia consistente e uma compreensão mais profunda do repertório comportamental desta espécie. Por exemplo, estudos sobre respostas ao estresse, aprendizado e memória, e comportamentos sociais e agressivos em peixe-zebra estão revelando paralelos surpreendentes com os mecanismos subjacentes aos comportamentos humanos. A padronização das metodologias e a clareza na descrição dos comportamentos são essenciais para garantir a reprodutibilidade e comparabilidade dos resultados entre diferentes laboratórios e estudos. Com o avanço das tecnologias de imagem e a integração de abordagens multiômicas, o uso do peixe-zebra em pesquisa translacional promete continuar crescendo, fornecendo um modelo valioso para desvendar os mistérios do cérebro e desenvolver novas terapias para doenças neurológicas. Para a execução do projeto, o rastreamento foi realizado analisando um peixe por vez.

3 METODOLOGIA

Este capítulo descreve detalhadamente todas as etapas do trabalho, conforme ilustrado na Figura 9, abrangendo desde a definição dos dados necessários para o desenvolvimento do projeto até a implementação do código de rastreamento. Cada etapa recebeu uma seção específica para facilitar a compreensão. O projeto foi organizado em quatro etapas principais. A primeira consiste na coleta de dados. A segunda no treinamento da rede *U-Net*. A terceira consiste na aplicação do modelo treinado para normalizar a iluminação do vídeo. E a quarta consiste na execução do rastreamento do peixe.

Figura 9 - Fluxograma do projeto



Fonte: Autor.

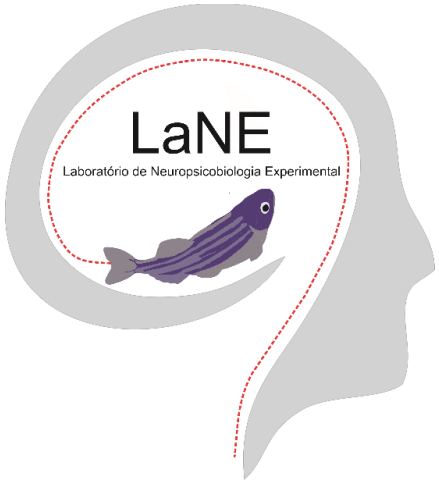
3.1 Coleta de dados

Nesta seção, será apresentada a metodologia utilizada para a coleta de dados.

3.1.1 Colaboração com o Laboratório LANE

Para a coleta de dados, trabalhou-se em conjunto com o laboratório LANE (Figura 10). A colaboração estreita com o LANE permitiu o recebimento de uma ampla variedade de vídeos contendo peixes-zebra em diferentes condições experimentais.

Figura 10 - Logo LANE.



Fonte: Laboratório de Neuropsicobiologia Experimental.

3.1.2 Critérios de seleção dos vídeos

Os vídeos foram selecionados de forma a capturar diversos comportamentos e interações dos peixes-zebra, levando em consideração fatores como iluminação, densidade de peixes e complexidade do fundo do aquário. O principal critério para a seleção dos vídeos foi a consistência na iluminação. A variação de iluminação é um fator crítico porque cada mudança nas condições de luz exige o ajuste de novos parâmetros nos algoritmos de rastreamento, o que aumenta significativamente a complexidade do processamento. Portanto, foram priorizados vídeos com iluminação uniforme e constante.

3.1.3 Desafios na consistência da iluminação

Durante o período de gravação, o laboratório LANE ainda não dispunha de um ambiente controlado que pudesse garantir a ausência de variações na iluminação. Esse desafio foi agravado pela claridade do sol que entrava pelas janelas, afetando a qualidade de alguns vídeos. Essas variações de luz natural introduziram inconsistências que precisaram ser cuidadosamente gerenciadas durante o processamento dos vídeos.

3.2 Treinamento da *U-Net* para normalização de iluminação

3.2.1 Preparação dos dados

O *dataset* foi elaborado a partir de um vídeo de referência, fornecido pelo laboratório LANE, que apresentava condições ideais de rastreamento, servindo como base para a extração de imagens. esse vídeo, foram extraídos 1.000 *frame*s originais, considerados como imagens de referência e armazenados na pasta *target_dataset*. Esses *frame*s, em condição ideal de iluminação, foram utilizados como ***target*** durante o treinamento do modelo.

A partir desse conjunto inicial, foi criado o diretório *input_dataset*, contendo versões degradadas artificialmente a partir dos *frame*s originais. Para gerar essas imagens, aplicaram-se transformações de brilho e saturação no espaço *HSV*, combinadas a um leve desfoque gaussiano em alguns casos. O processo foi automatizado pelo código de pré-processamento, que percorreu os *frame*s originais e, para cada um deles, produziu múltiplas variações com valores de:

- **Brilho:** entre 0,90–0,99 (subexposição leve) e 1,20–1,30 (superexposição moderada);
- **Saturação:** variação de ±10%, para simular alterações de cor comuns em condições de luz artificiais;
- **Desfoque:** aplicação opcional de *Kernel* gaussiano 3×3, para adicionar perda de nitidez.

Esse procedimento gerou 12 imagens degradadas por *frame* original, resultando em um total de 12.000 *frame*s no *input_dataset*, sempre preservando o pareamento direto com os respectivos *target*s em *target_dataset*. Assim, para cada entrada degradada, havia um *target* correspondente sem distorções, formando os pares necessários para o aprendizado supervisionado.

3.2.2 Pré-processamento

As imagens do *dataset* foram redimensionadas para 192 × 192 pixels, valor definido de forma a reduzir o custo computacional durante o treinamento da rede, mantendo, entretanto, informações visuais suficientes para que a rede pudesse aprender a distinguir as variações de iluminação e preservar a estrutura do peixe. Essa escolha é comum em aplicações de visão computacional que envolvem redes convolucionais, pois dimensões menores permitem maior rapidez no processamento e menor demanda de memória, sem comprometer de forma significativa a acurácia do modelo (ZHOU et al., 2020). Além disso, todas as imagens foram normalizadas para a faixa [0,1], procedimento fundamental em redes neurais, pois garante que os valores de entrada tenham uma escala uniforme, evitando a predominância de pixels com intensidades mais altas e permitindo maior estabilidade no cálculo dos gradientes durante o processo de *backpropagation*. A normalização também contribui para uma convergência mais rápida e estável do treinamento (LECUN et al., 2015). Após esses ajustes, o *dataset* foi dividido em três subconjuntos: treinamento (70%), validação (20%) e teste (10%), garantindo tanto a aprendizagem adequada do modelo quanto a avaliação imparcial de sua capacidade de generalização.

3.2.3 Arquitetura do modelo UNET

O modelo adotado para normalização de iluminação foi a *U-Net*, composta por caminhos de contração e expansão com conexões de salto que preservam detalhes espaciais essenciais para reconstrução de imagens.

Para o treinamento, foram definidos os seguintes parâmetros principais:

- **Filtros convolucionais:** 64 no primeiro nível, dobrando a cada nível subsequente, garantindo extração eficiente de características sem sobrecarregar a memória.
- **Tamanho das imagens:** 192×192 pixels, reduzindo o custo computacional e acelerando o treinamento, mantendo os detalhes do peixe necessários para o rastreamento.
- ***Batch size*:** 8 imagens por iteração, compatível com a *GPU* disponível e suficiente para estabilidade do treinamento.
- **Função de perda:** L1 *Loss*, escolhida por promover reconstruções mais precisas e evitar suavizações excessivas.

- **Otimizador:** Adam, com taxa de aprendizado de 1e-3, para atualização adaptativa dos pesos da rede.
- **Número de épocas:** 15, com *early stopping* após 3 épocas sem melhoria na validação, prevenindo *overfitting*.

Essas escolhas permitiram equilibrar a capacidade de reconstrução do modelo, o tempo de treinamento e as limitações computacionais disponíveis.

3.2.4 Avaliação do desempenho do modelo

Para avaliar o desempenho do modelo de normalização de iluminação, foram utilizadas duas métricas amplamente empregadas em reconstrução e restauração de imagens:

- *PSNR (Peak Signal-to-Noise Ratio)*: quantifica a qualidade da reconstrução comparando a imagem gerada pelo modelo com a imagem de referência (*target*). Essa métrica é sensível a diferenças de pixel, sendo útil para identificar alterações na intensidade e ruídos introduzidos durante a normalização (Wang et al., 2020).
- *SSIM (Structural Similarity Index)*: mede a similaridade estrutural entre a imagem normalizada e o *target*, avaliando a preservação de texturas, padrões e contraste local. Essa métrica é considerada mais próxima da percepção visual humana, permitindo identificar se a normalização manteve a aparência natural do objeto, neste caso, o peixe (Zhou et al., 2021).

A escolha dessas métricas se justifica pela necessidade de garantir tanto a fidelidade de pixel quanto a preservação das características estruturais das imagens, fatores essenciais para o correto rastreamento do peixe em etapas subsequentes do processamento.

Durante a avaliação do modelo, os valores de *PSNR* e *SSIM* foram calculados para os subconjuntos de validação e teste, permitindo uma análise objetiva do desempenho da normalização sob diferentes condições de iluminação. Além dos resultados numéricos, foram incluídas ilustrações representativas do funcionamento do modelo, apresentando casos com luminosidade aumentada e reduzida, de forma a demonstrar visualmente como a *U-Net* atua no processo de padronização.

3.3 Normalização do vídeo com *U-Net*

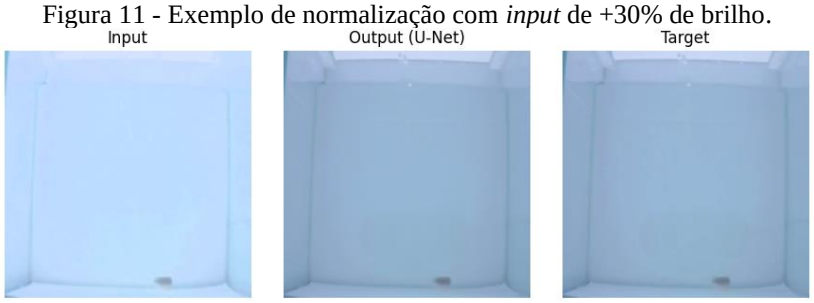
A normalização de vídeo foi implementada como um módulo independente, separado tanto do treinamento quanto do rastreamento, com o objetivo de otimizar a execução do projeto e aumentar a flexibilidade do sistema. A separação do treinamento permite que o modelo *U-Net* seja aplicado diretamente aos vídeos sem necessidade de re-treinamento a cada execução, garantindo consistência na normalização entre diferentes conjuntos de dados e reduzindo o tempo de processamento.

A independência em relação ao rastreamento possibilita utilizar o normalizador apenas quando necessário, evitando a modificação de *frame*s que já apresentam iluminação adequada e permitindo que vídeos com iluminação constante sejam analisados diretamente pelo módulo de rastreamento.

Durante a aplicação, cada *frame* é processado pelo modelo *U-Net*, que realiza a padronização da iluminação preservando as características cromáticas e estruturais do peixe. Essa abordagem assegura que variações de brilho não comprometam a detecção e o rastreamento subsequente, mantendo a precisão do método mesmo em condições de iluminação variáveis.

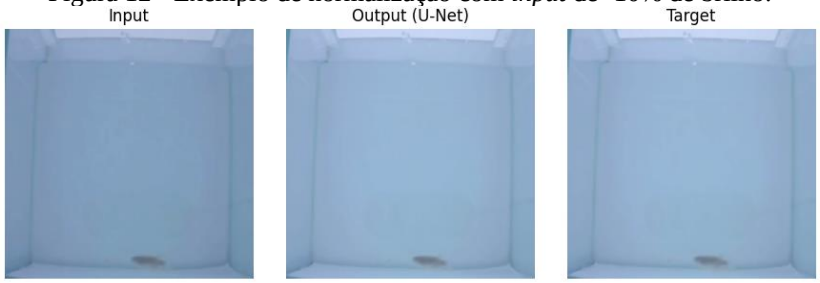
Nas figuras, são mostrados o *frame* original (*input*), o resultado obtido pelo modelo (*output*) e a imagem de referência (*target*). A Figura 11, demonstra o comportamento do modelo diante de um cenário artificialmente mais claro (+30% de brilho), ao passo que a Figura 12 exibe uma situação oposta, com redução de 10%.

Figura 11 - Exemplo de normalização com *input* de +30% de brilho.



Fonte: Autor.

Figura 12 - Exemplo de normalização com *input* de -10% de brilho.

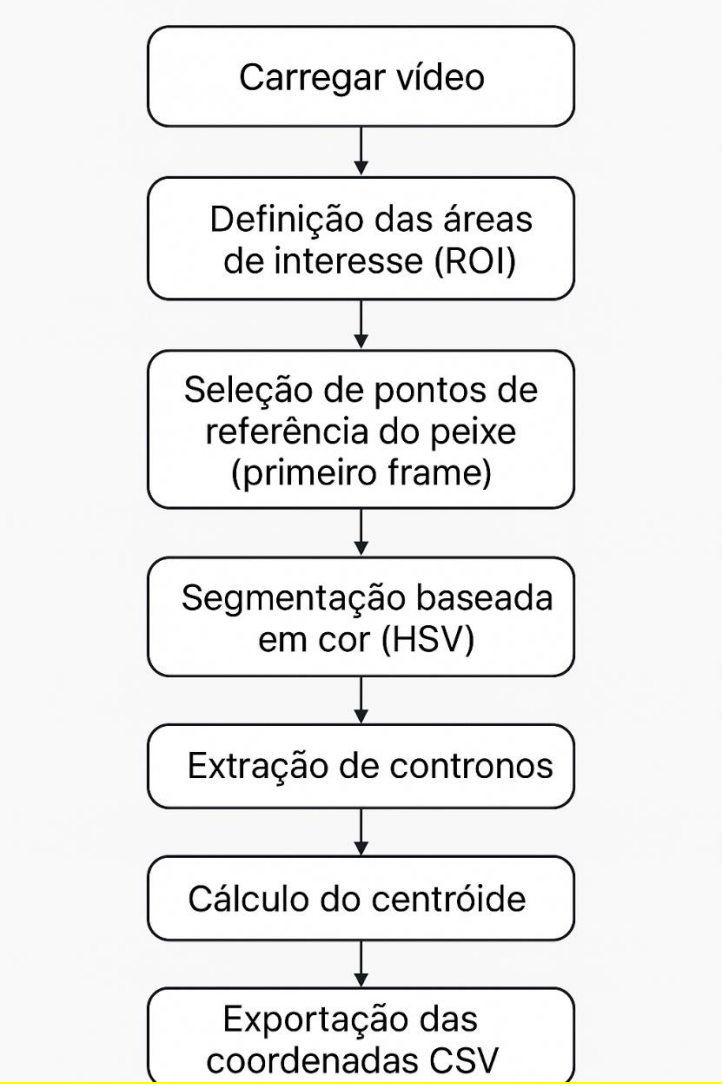


Fonte: Autor.

3.4 Implementação do *tracking*

Nesta seção, será apresentado um detalhamento mais aprofundado do processo de rastreamento. A Figura 13 ilustra o fluxograma da implementação do algoritmo, destacando a sequência das etapas realizadas.

Figura 13 – Fluxograma algoritmo.



Fonte: Autor.

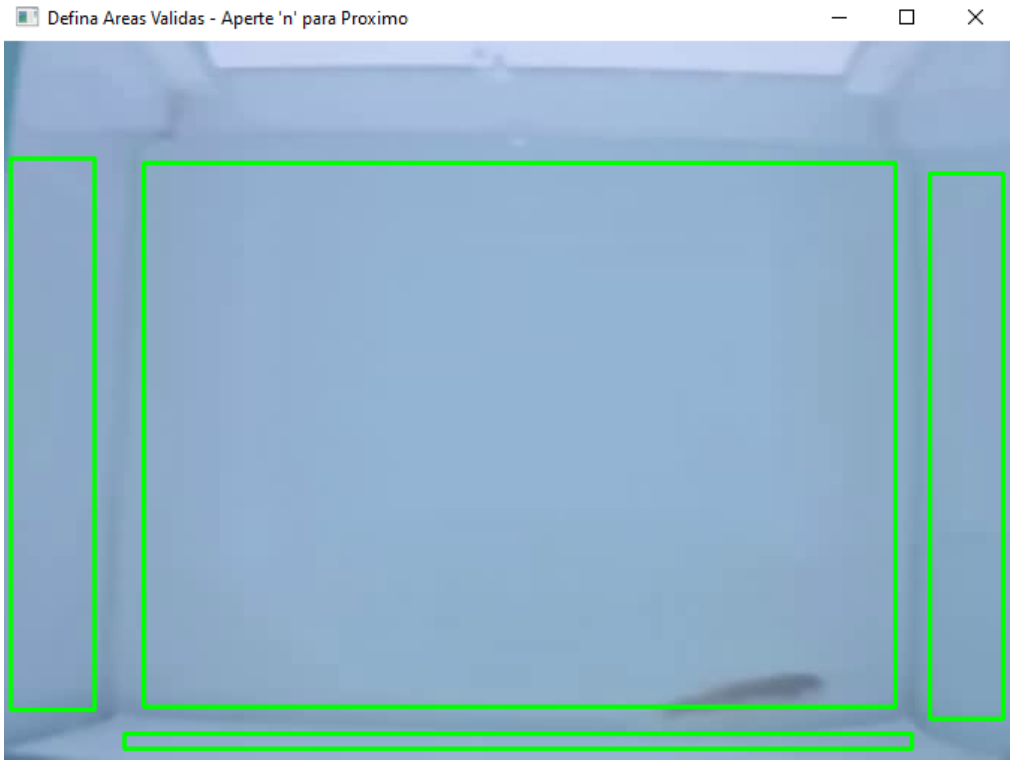
3.4.1 Definição de Áreas de Interesse (*ROI*)

A seleção da *ROI* (*Region of Interest*) consiste em delimitar de forma explícita as regiões do *frame* em que o peixe pode ser rastreado, com o objetivo de aumentar a eficiência do algoritmo e reduzir a ocorrência de falsos positivos. Essa etapa é fundamental para evitar que o

rastreador considere áreas com coloração semelhante à do peixe, como reflexos, bolhas ou junções entre os vidros do aquário, que poderiam comprometer a precisão da detecção.

A definição do *ROI* é realizada manualmente sobre o primeiro *frame* do vídeo, por meio de uma interface gráfica que permite ao usuário desenhar retângulos delimitando as regiões válidas. A partir dessa seleção, uma máscara binária é gerada, na qual os pixels correspondentes às áreas de interesse recebem valor 255 e os demais recebem valor 0. Essa máscara é aplicada a todos os *frame*s subsequentes, restringindo o processamento apenas às regiões definidas. A integração da máscara de *ROI* com a segmentação por cor no espaço *HSV* permite que apenas os pixels dentro das regiões válidas sejam analisados para detecção e rastreamento do peixe. Essa abordagem aumenta a robustez do método frente a interferências do fundo e melhora o desempenho computacional, pois reduz a quantidade de pixels a serem processados em cada *frame*. A Figura 14 ilustra a aplicação das *ROI*s sobre um *frame* do vídeo, evidenciando as áreas delimitadas para rastreamento.

Figura 14 - Definição de áreas de interesse.



Fonte: Autor.

3.4.2 Seleção de pontos de referência do peixe

A seleção de pontos de referência no peixe consiste em identificar manualmente pontos característicos do corpo do animal no primeiro *frame* do vídeo, com o objetivo de determinar a

faixa de cores no espaço *HSV* utilizada para o rastreamento. No trabalho anterior, essa faixa de cor era fixada diretamente no código, porém observou-se que, devido à instabilidade da iluminação entre vídeos, era necessário ajustar manualmente os valores a cada nova gravação.

Para contornar essa limitação, o presente método permite selecionar interativamente sete pontos distribuídos ao longo do corpo do peixe no primeiro *frame*. Os valores de *Hue* (H), *Saturation* (S) e *Value* (V) de cada ponto são extraídos e utilizados para definir os limites mínimo e máximo de cada canal *HSV*, que servirão como faixa de cor inicial para segmentação do peixe nos *frame*s subsequentes.

Essa abordagem automatiza a adaptação à variação de iluminação de cada vídeo, mantendo a segmentação específica do peixe sem a necessidade de redefinir manualmente as faixas de cor para cada gravação. A Figura 15 ilustra a seleção dos pontos de referência sobre o corpo do peixe.

Figura 15 - Seleção dos pontos.



Fonte: Autor.

3.4.3 Segmentação Baseada em Cor (*HSV*)

A segmentação baseada em cor é a etapa central no rastreamento do peixe-zebra, responsável por isolar o objeto de interesse do fundo do vídeo. Neste trabalho, a segmentação

foi realizada no espaço de cores *HSV* (*Hue*, *Saturation*, *Value*), uma escolha estratégica devido à maior separação entre informações de cor e brilho, o que permite maior robustez frente a variações de iluminação.

Os limites da faixa *HSV* são definidos a partir dos sete pontos de referência selecionados manualmente no primeiro *frame* do vídeo. Para cada canal (H, S e V), são determinados os valores mínimos e máximos observados nos pontos, aos quais são adicionadas margens pré-definidas (10 para *Hue*, 30 para *Saturation* e 30 para *Value*). Essa faixa adaptativa garante que pequenas variações de cor ou iluminação entre os *frame*s não comprometam a segmentação. Além disso, os valores mínimo e máximo são restringidos aos limites válidos do espaço *HSV* no *OpenCV* (*Hue*: 0–179; *Saturation* e *Value*: 0–255).

A segmentação é aplicada a cada *frame* do vídeo por meio da função *cv2.inRange()*, que gera uma máscara binária indicando os pixels que pertencem à faixa de cor definida. Essa máscara é posteriormente combinada com a máscara da *ROI*, de forma que apenas os pixels localizados em regiões válidas e dentro da faixa de cor do peixe sejam considerados. Para reduzir ruídos, aplica-se um filtro de mediana à máscara resultante, suavizando pequenas imperfeições e partículas isoladas que possam interferir na detecção.

O resultado da segmentação é uma máscara binária em que os pixels correspondentes ao peixe são destacados, permitindo a detecção de contornos e o cálculo do centroide do animal. A Figura 16 ilustra um exemplo de segmentação por cor aplicada a um *frame* do vídeo, evidenciando a separação entre o peixe e o fundo do aquário.

Figura 16 - Aplicação da máscara binária.



Fonte: Autor.

3.4.4 Extração de contornos e centroide

Após a segmentação do peixe-zebra por cor no espaço *HSV*, a próxima etapa consiste na extração dos contornos e cálculo do centroide, que permite determinar a posição do animal em cada *frame*. Essa etapa é realizada sobre a máscara binária resultante da segmentação, na qual os pixels correspondentes ao peixe são destacados e o fundo é representado em preto.

Os contornos são identificados utilizando a função *cv2.findContours()*, que detecta todos os conjuntos de pixels conectados dentro da máscara. Dentre os contornos detectados, seleciona-se aquele com maior área, assumindo que corresponde ao peixe, enquanto contornos menores, resultantes de ruídos ou partículas isoladas, são descartados.

Para cada contorno selecionado, são calculados os momentos espaciais, que são medidas estatísticas dos pixels do contorno. O centroide do peixe (C_x, C_y) é obtido diretamente a partir dos momentos de primeira ordem:

$$C_x = \frac{M_{10}}{M_{00}}, \qquad C_Y = \frac{M_{01}}{M_{00}}$$

onde M_{00} representa a área do contorno, e M_{10} e M_{01} correspondem às somas ponderadas das coordenadas *x* e *y* dos pixels do contorno, respectivamente.

O centroide é então marcado no *frame* original para visualização, permitindo acompanhar o movimento do peixe ao longo do vídeo.

3.4.5 Exportação dos dados

Para possibilitar análises quantitativas do movimento do peixe, as coordenadas do centroide calculadas em cada *frame* são exportadas para um arquivo no formato *CSV* (*Comma-Separated Values*). Esse formato é amplamente utilizado em análise de dados, pois permite fácil integração com *Softwares* estatísticos, planilhas e linguagens de programação, como *Python* e *R*.

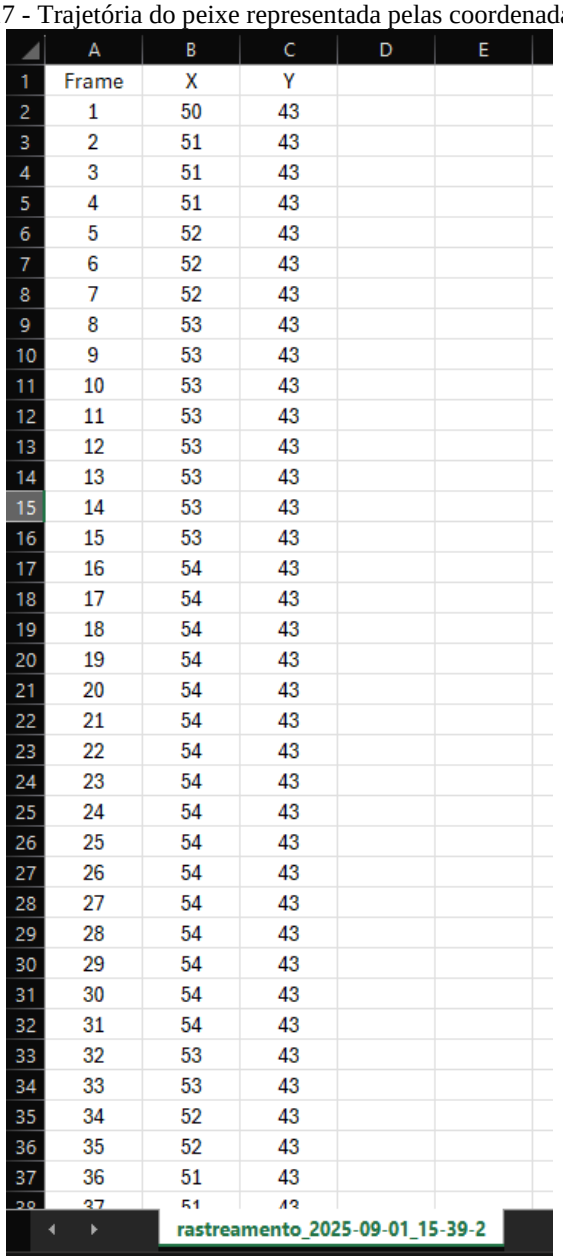
Durante o rastreamento, para cada *frame* em que o centroide do peixe é detectado, são registradas as seguintes informações:

1. Número do *frame* no vídeo.
2. Coordenada X do centroide.
3. Coordenada Y do centroide.

A exportação é realizada em tempo real dentro do *loop* de processamento de vídeo, garantindo que cada detecção válida seja imediatamente escrita no arquivo *CSV*. *Frame*s sem detecção também são contabilizados, mas não possuem coordenadas registradas, permitindo avaliar a taxa de sucesso do rastreamento.

O arquivo *CSV* gerado possui um cabeçalho indicando cada coluna, facilitando a interpretação dos dados e a utilização em análises subsequentes, como cálculo de trajetórias, velocidades instantâneas e padrões comportamentais do peixe, conforme Figura 17.

Figura 17 - Trajetória do peixe representada pelas coordenadas x e y.



	A	B	C	D	E
1	Frame	X	Y		
2	1	50	43		
3	2	51	43		
4	3	51	43		
5	4	51	43		
6	5	52	43		
7	6	52	43		
8	7	52	43		
9	8	53	43		
10	9	53	43		
11	10	53	43		
12	11	53	43		
13	12	53	43		
14	13	53	43		
15	14	53	43		
16	15	53	43		
17	16	54	43		
18	17	54	43		
19	18	54	43		
20	19	54	43		
21	20	54	43		
22	21	54	43		
23	22	54	43		
24	23	54	43		
25	24	54	43		
26	25	54	43		
27	26	54	43		
28	27	54	43		
29	28	54	43		
30	29	54	43		
31	30	54	43		
32	31	54	43		
33	32	53	43		
34	33	53	43		
35	34	52	43		
36	35	52	43		
37	36	51	43		

Fonte: Autor.

4 RESULTADOS E DISCUSSÕES

Neste capitulo serão apresentados os resultados obtidos durante o projeto.

4.1 Testes iniciais com algoritmos de *tracking* da biblioteca *OpenCV*

Nos primeiros testes realizados com algoritmos de *tracking* prontos disponíveis na biblioteca *OpenCV*, foram avaliados os métodos *Boosting*, *MIL*, *KCF*, *TLD*, *CSRT*. Esses algoritmos utilizavam uma técnica de *tracking* baseada na seleção de uma área inicial, onde o rastreamento era feito a partir dessa área definida.

Dentre os algoritmos testados, o *CSRT* apresentou os melhores resultados em termos de precisão e consistência. No entanto, apesar de seu desempenho superior em comparação aos outros métodos, o *CSRT* ainda enfrentou dificuldades significativas durante os testes. Um dos principais problemas observados foi a perda do objeto rastreado após um certo tempo, especialmente quando o peixe se aproximava da borda do aquário, como observado na Figura 18. Essa limitação impactava negativamente a continuidade e a confiabilidade do rastreamento.

Figura 18 - *Tracking* utilizando *CSRT*.



Fonte: Autor.

Outro desafio identificado foi quando o peixe-zebra mudava de configuração, como ao ficar de frente para o aquário. Nesses casos, a mudança na aparência do peixe dificultava a detecção e o rastreamento precisos pelo algoritmo, resultando em falhas e interrupções no processo de *tracking*. Essas dificuldades indicaram a necessidade de desenvolver e ajustar métodos de rastreamento mais robustos e adaptáveis, capazes de lidar eficazmente com as variações de movimento e aparência dos peixes-zebra no ambiente de estudo.

4.1.1 Adição do filtro de *Kalman*

Como os resultados iniciais com os algoritmos de *tracking* do *OpenCV*, não atingiram as expectativas desejadas, foi decidido incorporar um Filtro de *Kalman* ao código do *CSRT* na tentativa de melhorar a precisão e a estabilidade do rastreamento. A implementação do Filtro de *Kalman* visava melhorar a capacidade do algoritmo de lidar com as mudanças abruptas na posição e na aparência dos peixes, além de reduzir a perda do objeto rastreado, especialmente quando o peixe se aproximava das bordas do aquário ou mudava de orientação. O filtro deveria atuar suavizando as trajetórias estimadas pelo *CSRT*, garantindo uma continuidade mais robusta no rastreamento. No entanto, apesar das expectativas, a adição do Filtro de *Kalman* não resultou em uma melhoria significativa nos resultados. Embora o filtro tenha contribuído para uma leve suavização nas trajetórias de movimento, ele não conseguiu resolver os problemas fundamentais observados no rastreamento inicial. O *CSRT*, mesmo com o suporte do Filtro de *Kalman*, ainda apresentava dificuldades para manter o rastreamento em situações onde o peixe mudava de aparência ao virar de frente para o aquário, além de continuar perdendo o objeto quando este chegava às bordas do aquário.

A utilização das bibliotecas prontas do *OpenCV* não proporcionou os resultados esperados. Em busca de uma solução alternativa, decidiu-se por trabalhar com filtros de cores. Para isso, foram testadas diferentes faixas de cores, especificamente os modelos de cor *HSV* e *BGR*.

4.2 Filtro de cor *BGR*

Inicialmente, foi testado um código de filtro *BGR* simples, que criava uma máscara e realizava o rastreamento (*tracking*) com base nos limites *BGR* definidos como parâmetros. Embora fosse um código simples, a simplicidade do código proporcionou uma base sólida para

futuras melhorias e otimizações no processamento das imagens, o *tracking* pode ser observado pela Figura 19.

Figura 19 – *Tracking BGR*.



Fonte: Autor.

Após a realização de diversos testes com filtros de Kalman e variações de filtros de Kernel, foi implementado o código descrito anteriormente, que apresentou melhor desempenho no rastreamento em comparação com os outros métodos testados (POSSEBON, 2024). Além de aumentar a precisão do rastreamento, o algoritmo conseguiu lidar de forma mais eficaz com ruídos externos, como variações de iluminação.

O tempo de processamento do projeto é praticamente instantâneo em um notebook com processador Intel Core i7 de 7ª geração, mesmo sem placa de vídeo dedicada, o que demonstra a leveza do código. A cada 10 *frame*s, o código imprime as coordenadas do peixe em pixels para o usuário.

A Figura 20 apresenta um *frame* do código implementado na prática, destacando o ponto vermelho que representa o rastreamento do peixe.

Figura 20 – *Tracking BGR* utilizando centróide.



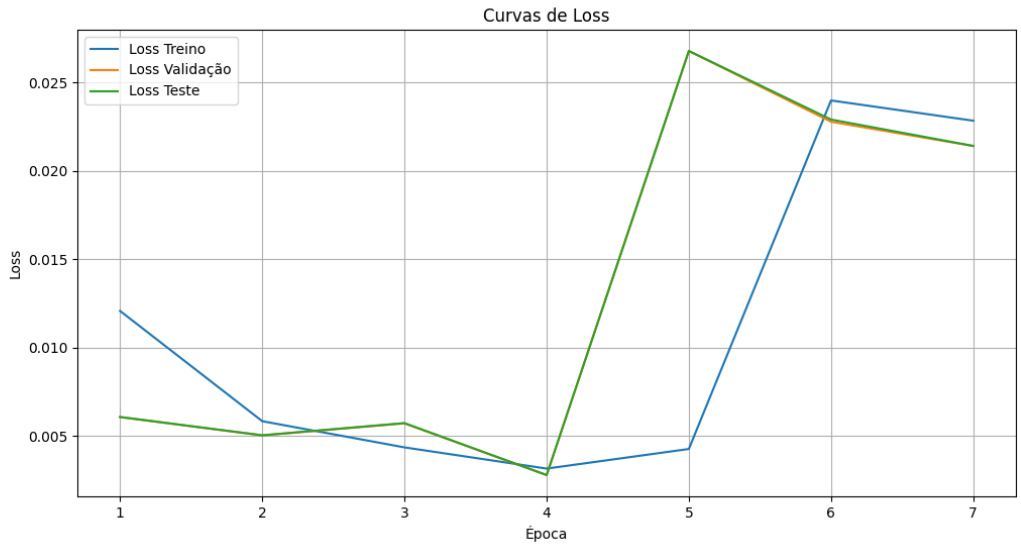
Fonte: Autor.

O rastreamento em *BGR* apresentou resultados satisfatórios na aplicação inicial. Entretanto, as análises realizadas demonstraram que o método em *HSV* é mais robusto diante de variações de iluminação. Para aprimorar a eficiência do processo, adotou-se a utilização da arquitetura *U-Net*, cujo objetivo é mitigar as variações de iluminação e potencializar o desempenho do rastreamento em *HSV*.

4.3 Arquitetura *U-Net*

A *U-Net* foi treinada com o objetivo de normalizar a iluminação nos *frame*s, reduzindo variações indesejadas e preservando as características visuais do peixe necessárias para o rastreamento. A evolução das perdas de treino, validação e teste pode ser observada na Figura 21.

Figura 21 - Curvas de *loss* da arquitetura *U-Net* durante o processo de treinamento, validação e teste.



Fonte: Autor.

Observa-se que, nas primeiras épocas, o modelo apresentou uma redução consistente das perdas, convergindo rapidamente para valores baixos. A partir da quinta época, houve um aumento abrupto nas curvas de *loss*, tanto em validação quanto em teste, indicando um possível ajuste sensível dos parâmetros de aprendizado nesse ponto. Apesar dessa oscilação, as curvas voltaram a se estabilizar, mantendo-se próximas entre si até o final do treinamento.

Esse comportamento sugere que o modelo não sofreu *overfitting* severo, já que as curvas de treino, validação e teste acompanharam tendências semelhantes. Além disso, mesmo com oscilações, os valores finais de *loss* permaneceram baixos e próximos entre os diferentes conjuntos, o que evidencia uma boa capacidade de generalização da rede. As métricas quantitativas finais estão resumidas na Tabela 1.

Tabela 1 - Métricas de desempenho do modelo *U-Net*.

Loss Treino	Loss Validação	Loss Teste	*PSNR* Médio	*SSIM* Médio
0.0228	0.0214	0.0214	30.66	0.919

Fonte: Autor.

O PSNR médio de aproximadamente 30 dB confirma que a *U-Net* produziu imagens com boa qualidade em relação às referências, enquanto o SSIM médio de 0,91 demonstra preservação estrutural consistente, garantindo que os detalhes relevantes do peixe e do aquário não foram comprometidos.

4.4 Rastreamento

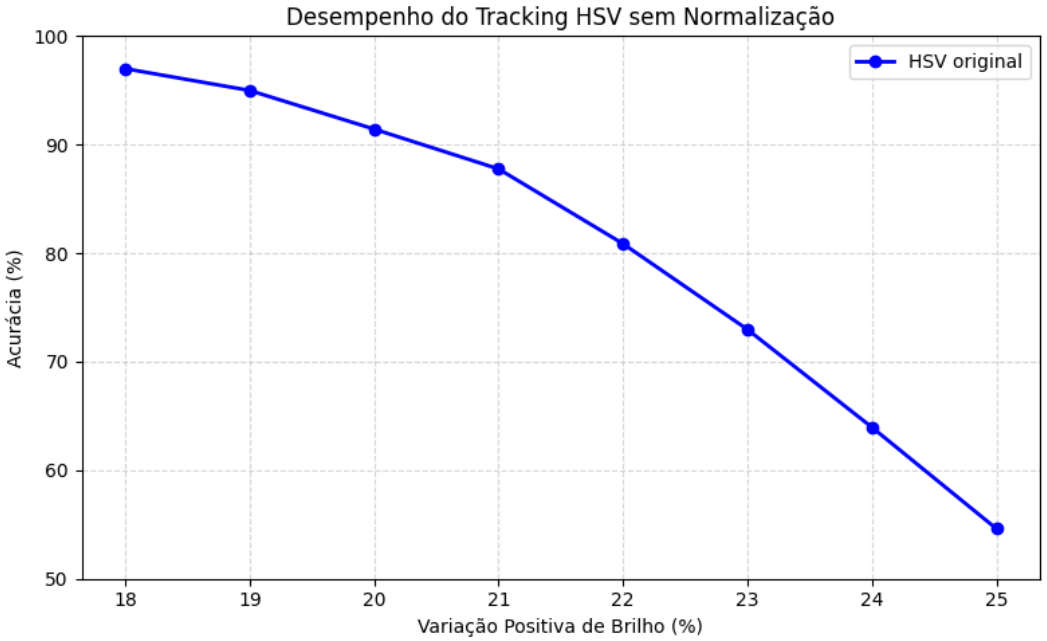
A etapa de rastreamento foi avaliada em diferentes condições de iluminação, comparando-se os desempenhos obtidos sem normalização e com a aplicação da *U-Net* para

normalização de iluminação. Para esta análise, foram consideradas métricas de acurácia, falsos positivos, *frame*s não detectados e deslocamento do centroide entre *frame*s consecutivos.

4.4.1 Rastreamento sem normalização

A Figura 22 apresenta a relação entre a variação do brilho positivo aplicado ao vídeo e a acurácia do rastreamento em *HSV*. Nos testes com aumento de brilho, o método obteve 97% de acurácia com brilho de 18%. À medida que o brilho foi elevado progressivamente, houve uma queda constante no desempenho: 94,99% em brilho 19% até 54,62% em brilho 25%. Esse resultado demonstra que o algoritmo em *HSV* é altamente sensível ao excesso de iluminação, que reduz o contraste entre o peixe e o fundo do aquário, prejudicando a detecção.

Figura 22 - Desempenho do *Tracking HSV* com variação positiva de brilho sem normalização.

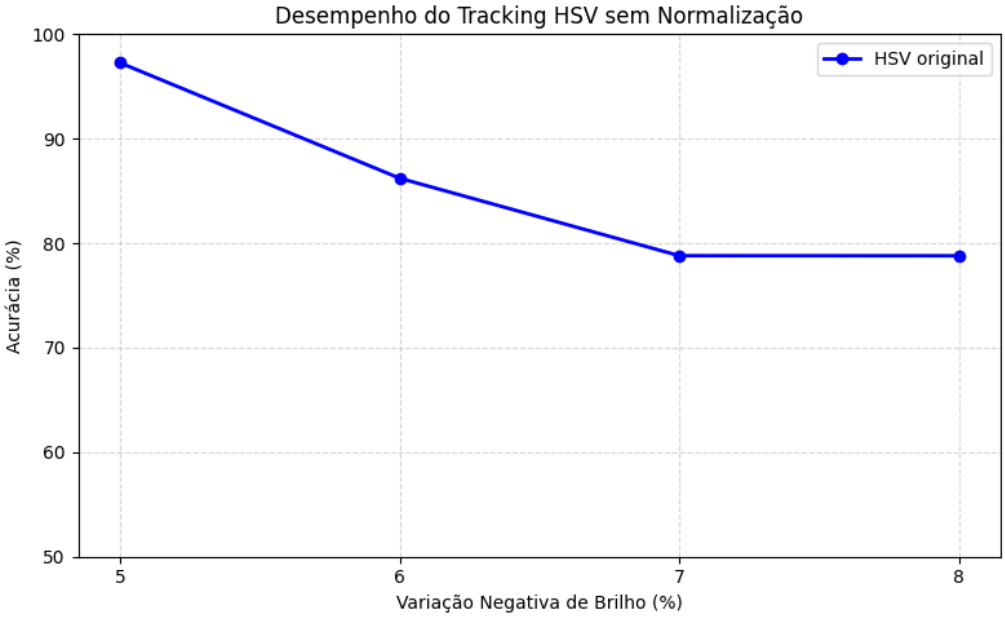


Fonte: Autor.

Nos testes com redução do brilho, a curva de acurácia mostrou inicialmente valores satisfatórios, chegando a 97,26% com 5% de redução. Contudo, a partir desse ponto, a queda foi acentuada, atingindo 86,22% com 6% de redução e 78,81% com 7% de redução. Já com 8% de redução, a acurácia registrada foi de 78,8%, valor semelhante ao anterior. Porém, essa aparente estabilidade não reflete um bom desempenho: nesse nível de iluminação, surgem manchas escuras no fundo do aquário com coloração muito próxima à faixa *HSV* do peixe. Assim, o rastreador mantém uma taxa de detecção alta, mas a maior parte dos pixels identificados corresponde a falsos positivos. O algoritmo não consegue distinguir essas

manchas do peixe real e, portanto, os resultados deixam de ser confiáveis mesmo sem queda brusca no percentual. A Figura 23 representa a relação entre a variação de brilho negativa aplicada ao vídeo e a acurácia do rastreamento em *HSV*.

Figura 23 - Desempenho do *Tracking HSV* com variação negativa de brilho sem normalização.



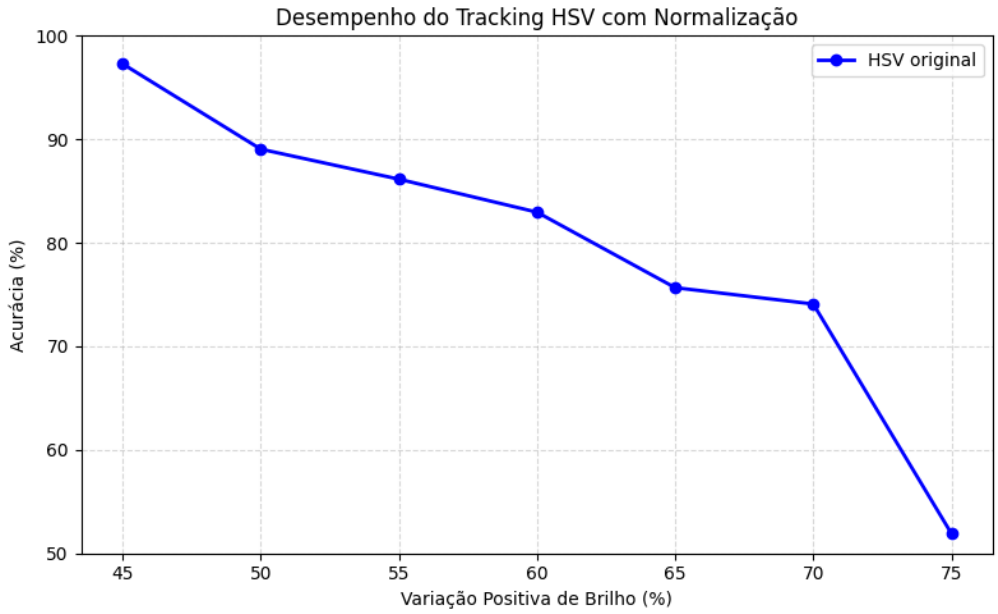
Fonte: Autor.

4.4.2 Rastreamento *HSV* com normalização *U-Net*

A Figura 24 apresenta a relação entre a variação do brilho positivo aplicado ao vídeo e a acurácia do rastreamento normalizado em *HSV*. Nos testes com aumento de brilho, o método obteve 97,29% de acurácia com brilho de 45%. À medida que o brilho foi elevado progressivamente, houve uma queda constante no desempenho: 89,05% em brilho 50%, 86,15% em 55%, 82,96% em 60%, 75,67% em 65%, 74,09% em 70%, até 51,92% em brilho 75%.

Esses resultados indicam que, mesmo após a normalização de iluminação, o algoritmo apresenta redução significativa de acurácia em condições de brilho extremo, embora mantenha desempenho elevado em níveis de brilho moderado. A normalização contribui para reduzir a sensibilidade do rastreamento a variações de iluminação, preservando o contraste entre o peixe e o fundo do aquário em condições mais favoráveis, mas não é capaz de compensar totalmente efeitos de saturação e perda de detalhes em brilhos muito altos.

Figura 24 - Desempenho do *Tracking HSV* com variação positiva de brilho com normalização.

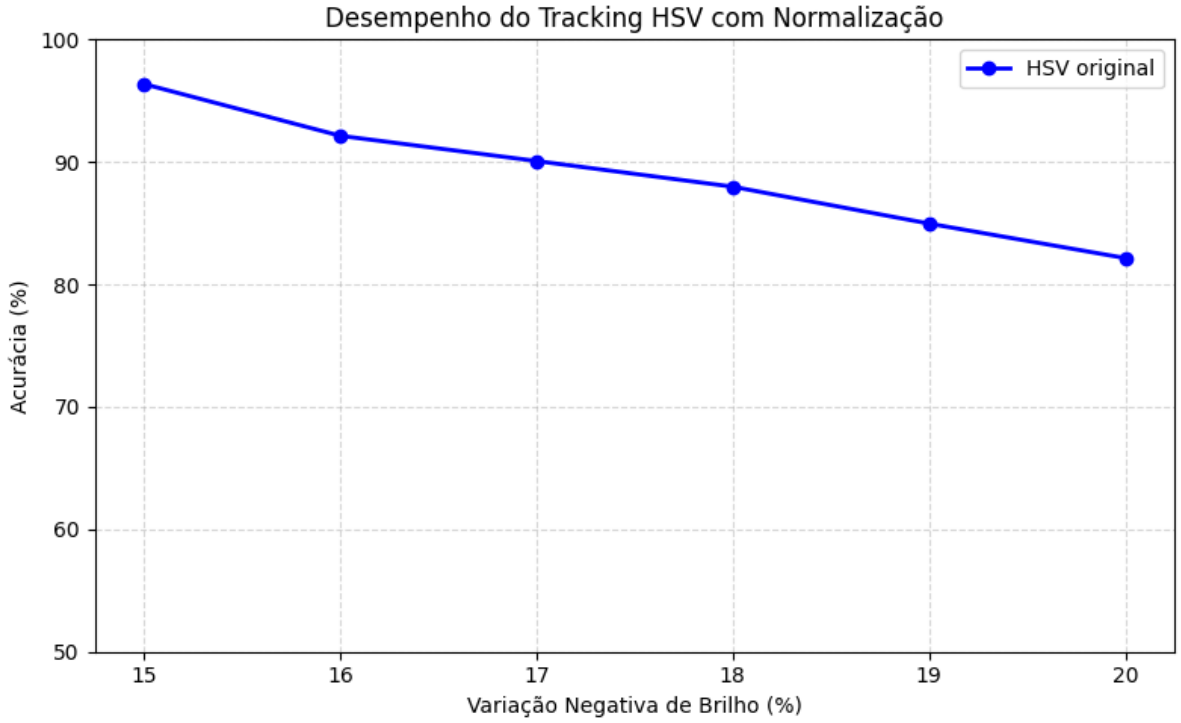


Fonte: Autor.

A representa a relação entre a variação de brilho negativa aplicada ao vídeo e a acurácia do rastreamento normalizado em *HSV*. Nos testes com redução de brilho, a curva de acurácia apresentou inicialmente valores satisfatórios, alcançando 96,37% com brilho de 15%. À medida que o brilho diminuiu, houve uma queda progressiva no desempenho: 92,14% em 16%, 90,08% em 17%, 87,98% em 18% e 84,97% em 19%.

No entanto, com brilho de 20%, apesar da acurácia registrada ser de 82,15%, essa aparente estabilidade não reflete um desempenho confiável. Nessa condição de iluminação, surgem manchas escuras no fundo do aquário, com coloração muito próxima à faixa *HSV* do peixe. Como consequência, o rastreador passa a identificar grande parte dessas manchas como peixe, gerando falsos positivos. Dessa forma, embora o percentual de acurácia não apresente uma queda abrupta, os resultados deixam de ser representativos da detecção real do animal.

Figura 25 - Desempenho do *Tracking HSV* com variação positiva de brilho com normalização.



Fonte: Autor.

Na Figura 26, observa-se o vídeo com brilho 20% menor em relação ao *frame* inicial. Apesar de ter passado pelo pré-processamento de normalização de iluminação, as manchas não foram corrigidas e, consequentemente, o rastreador não conseguiu detectar o peixe.

Figura 26 - Falha de rastreamento causada por variação negativa de iluminação.

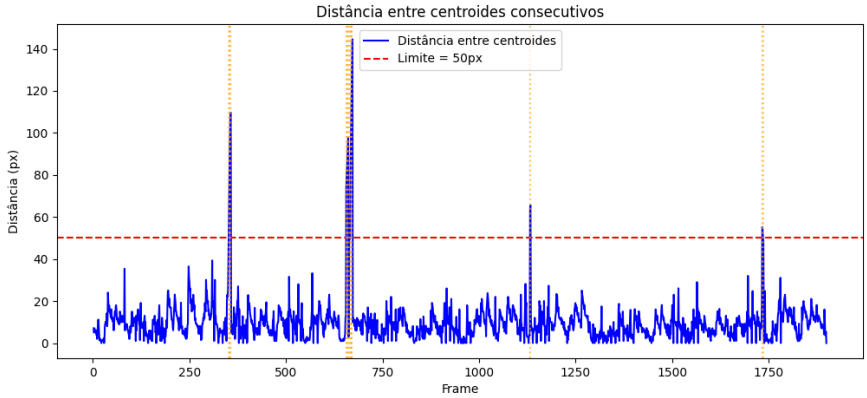


Fonte: Autor.

4.4.3 Estabilidade do *tracking*

Na Figura 27 é apresentado o gráfico da distância entre centroides de *frame*s consecutivos ao longo do vídeo. Nota-se uma curva com variações suaves, o que indica que o rastreamento foi consistente, com poucos saltos abruptos entre os pontos detectados. O vídeo analisado corresponde a um caso com aumento de 40% no brilho em relação ao *frame* inicial, tendo sido previamente submetido ao processo de normalização de iluminação, o que contribuiu para a estabilidade observada.

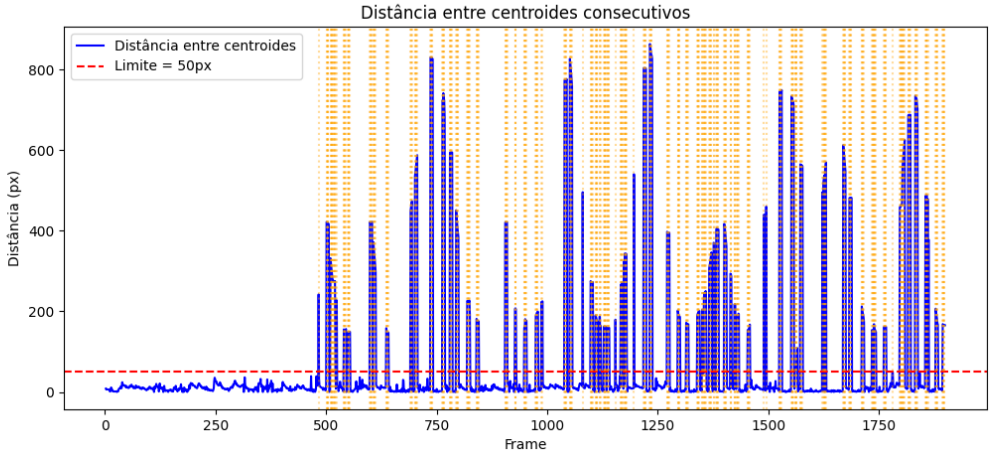
Figura 27 - Gráfico da distância entre *centroide* de *frame*s consecutivos ao longo do vídeo normalizado.



Fonte: Autor.

Na Figura 28, observa-se um vídeo em que o rastreamento não apresentou bom desempenho. Nota-se que o gráfico da distância entre *centroide* apresenta um número maior de picos, evidenciando a baixa precisão do rastreamento, que alternou de forma abrupta entre diferentes regiões da imagem. Esse comportamento é incoerente, pois a análise é feita quadro a quadro e, portanto, tais deslocamentos não correspondem ao movimento real do peixe. Em determinados trechos, a distância entre *centroides* consecutivos chega a aproximadamente 800 pixels em apenas um *frame*, valor incompatível com a realidade e caracterizado como um falso positivo.

Figura 28 - Gráfico da distância entre *centroide* em vídeo com falha de rastreamento.



Fonte: Autor.

5 CONCLUSÃO

Este trabalho teve como objetivo o desenvolvimento de um sistema de rastreamento automatizado do comportamento do peixe-zebra, com ênfase na mitigação dos efeitos causados por variações de iluminação no ambiente do laboratório LANE. Para isso, foi proposta a integração de uma rede neural convolucional do tipo *U-Net* como etapa de pré-processamento, responsável pela normalização da iluminação dos vídeos, seguida da aplicação de técnicas clássicas de rastreamento baseadas em segmentação por cor no espaço *HSV*. Os resultados obtidos demonstraram que a utilização da *U-Net* como ferramenta de normalização de iluminação contribuiu significativamente para a estabilidade e a confiabilidade do rastreamento. Quando comparado ao rastreamento realizado diretamente sobre os vídeos originais, o vídeo submetido ao processo de normalização apresentou uma redução expressiva de falhas de segmentação, menor ocorrência de perdas do objeto rastreado e trajetórias mais coerentes com o movimento real do peixe ao longo dos quadros.

A avaliação quantitativa do modelo de normalização reforça esses resultados, com valores médios de *PSNR* de aproximadamente 30,66 dB e *SSIM* de 0,919, indicando que a rede foi capaz de corrigir distorções de iluminação de forma eficaz, preservando tanto as características estruturais quanto as informações cromáticas relevantes para o rastreamento. Esses índices evidenciam que a normalização não comprometeu a integridade visual do peixe-zebra, favorecendo diretamente a etapa subsequente de segmentação por cor. De modo geral, a abordagem proposta mostrou-se eficaz ao combinar a robustez das redes neurais convolucionais com a simplicidade e o baixo custo computacional dos métodos clássicos de visão computacional. Essa estratégia híbrida possibilitou a obtenção de um desempenho satisfatório mesmo em cenários com iluminação adversa, mantendo a eficiência do rastreamento sem a necessidade de empregar soluções completamente baseadas em aprendizado profundo para a detecção do objeto, o que preserva a leveza computacional do sistema.

Como trabalhos futuros, propõe-se a ampliação do sistema para o rastreamento simultâneo de múltiplos indivíduos, possibilitando análises de comportamento social do peixe-zebra. Além disso, a adaptação do método para diferentes condições experimentais, como variações de cor do fundo do aquário, múltiplas espécies ou diferentes configurações de câmera, pode aumentar a generalização da abordagem. Outra possibilidade consiste na investigação de arquiteturas de aprendizado profundo mais leves ou especializadas para normalização de iluminação, bem como na integração de métricas comportamentais adicionais, como velocidade

média, aceleração e padrões espaciais de permanência, ampliando o potencial analítico do sistema para aplicações em estudos neurobiológicos e toxicológicos.

6 REFERÊNCIAS

Balaji, S. R., & Karthikeyan, S. (2017). **A survey on moving object *tracking* using image processing**. Coimbatore, Tamilnadu, India. 2024.

BALLARD, D.; BROWN, C.; **Computer Vision Prentice Hall**, 1982.

Bayat, A. (2002). **Science, medicine, and the future: Bioinformatics, BMJ**. Disponível em: https://sol.sbc.org.br/index.php/ersi-rj/article/view/10112/9988. Acesso 28 de mar. 2024.

CAB (2018). **What is Bioimaging**? http://bit.ly/2kAYPeZ, Acesso: 30 de Maio de 2024.

D. Koniar, L. Hargaš, Z. Loncová, F. Duchoň and P. Beňo. (2016). **Laboratory animals tracking in videosequences**. ELEKTRO, Strbske Pleso, Slovakia, 2016, pp. 537-542, doi: 10.1109/ELEKTRO.2016.7512134.

Gomes, J., Darsa, L., Costa, B., and Velho, L. (1999). **Warping and Morphing of Graphical Objects**. Morgan Kaufmann Publishers, San Francisco.

H. Schweitzer, J.W. Bell, and F. Wu, "**Very fast template matching**" European Conference on Computer Vision, pp. 358-372, 2002.

Iivarinen, J., Peura, M., Sarela, J., and Visa, A. (1997). **Comparison of combined shape descriptors for irregular objects**. In Clark, A. F., editor, BMVC97, volume II, pages 430–439.

Janku, P., Koplik, K., Dulik, T., & Szabo, I. (2016). ***Comparison of tracking algorithms implemented in OpenCV***. *MATEC Web of Conferences, 76, 04031*. doi:10.1051/matecconf/20167604031

Kalueff AV, Gebhardt M, Stewart AM, Cachat JM, Brimmer M, Chawla JS, Craddock C, Kyzar EJ, Roth A, Landsman S, Gaikwad S, Robinson K, Baatrup E, Tierney K, Shamchuk A, Norton W, *MIL*ler N, Nicolson T, Braubach O, Gilman CP, Pittman J, Rosemberg DB, Gerlai R, Echevarria D, Lamb E, Neuhauss SC, Weng W, Bally-Cuif L, Schneider H; **Zebrafish Neuroscience Research Consortium. Towards a comprehensive catalog of zebrafish behavior 1.0 and beyond**. Zebrafish. 2013. doi: 10.1089/zeb.2012.0861.

MARTTY, H. **Los peces y sus enfermedades**. Buenos Aires: Albatros, 1986. 2v.

Miller, Greg. (2018). **A New Research Tool That Can Track One Fish Among Many**, Wired. https://www.wired.com/2014/06/software-tracks-animals-science. Acesso em: 5 de Jun 2024.

M. Nixon, A. Aguado. **Feature Extraction & Image Processing**, [S.l.]: Elsevier, 2008.

Panadeiro, V., Rodriguez, A., Henry, J. *et al.* **A review of 28 free animal-*tracking* software applications: current features and limitations**. *Lab Anim* 50, 246–254 (2021). https://doi.org/10.1038/s41684-021-00811-1.

Pedrini, H. e Schwartz W. R. (2008) **Análise de Imagens digitais: Princípios, algoritmos e aplicações** - São Paulo: Thomson Learning.

Peng, H., Bateman, A., Valencia, A., Wren, J. D. (2012) **Bioimage informatics: a newcategory in Bioinformatics**. Bioinformatics, v. 28, n. 8, p.1057-1057.

POSSEBON, Diogo. Desenvolvimento de um algoritmo para rastreamento do comportamento do peixe-zebra em ambiente de laboratório. 2024. Projeto Integrador – Curso de Engenharia de Controle e Automação, Universidade Federal de Santa Maria, Santa Maria, 2024.

REALI COSTA, A. H.; PEGORARO, R.; STOLFI, G.; SICHMAN, J.;PAIT, F. M.; FERASOLI FILHO, H.; GUARANÁ. **Robot-Soccer Team: Some Architectural Issues**. Anais do 4 O Simpósio Brasileiro de Automação Inteligente, São Paulo, setembro, 1999.

RODRIGUES, Jorge. **Análise da variação dinâmica da complexidade de um algoritmo de *tracking***. 2012.

Santos Silva, A., Marcolino Quintao Severgnini, F., Lopes Oliveira, M., Matheus Santiago Mendes, V., & Assis Peixoto, Z. M. (2016). **Object *Tracking* by Color and Active Contour Models Segmentation.** IEEE Latin America Transactions, 14(3), 1488–1493.

Soleimanitaleb, Z., Keyvanrad, M. A., & Jafari, A. (2019). **Object *Tracking* Methods:A Review**. 2019 9th International Conference on Computer and Knowledge Engineering (ICCKE). doi:10.1109/iccke48569.2019.8964761

SZELISKI, Richard. **Computer Vision: Algorithms and Applications**. 2. Ed. The University of Washington. 2022.